A

MAJOR PROJECT REPORT

ON

SKILL CREDENTIALING SYSTEM

Submitted in partial fulfillment of the requirement for the award of the degree of

B. TECH

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By:

22M91A05B3 - RAKESH GAJRE

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

AURORA'S SCIENTIFIC AND TECHNOLOGICAL INSTITUTE

Approved by AICTE, affiliated JNTU Hyderabad GHATKESAR -501301, TELANGANA

[2025-2026]

AURORA'S SCIENTIFIC AND TECHNOLOGICAL INSTITUTE

Approved by AICTE, affiliated JNTU Hyderabad GHATKESAR -501301, TELANGANA

[2025-2026]

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Major Project titled **"SKILL CREDENTIALING SYSTEM"** is a Bonafide work conducted during the IV/II semester by **RAKESH GAJRE -22M91A05B3,** This work has been completed in partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science & Engineering at **AURORA'S SCIENTIFIC AND TECHNOLOGICAL INSTITUTE**, affiliated with **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD**.

PROJECT GUIDE

Hima Bhindhu

HEAD OF DEPARTMENT

Dr. M. Sridhar

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

The completion of our Major Project presents us with the opportunity to express our heartfelt gratitude to everyone who supported us throughout this journey.

First and foremost, we express our profound gratitude to the Almighty for the spiritual guidance and blessings that enabled us to complete this Major Project. We are deeply thankful to our parents for their unwavering support in our pursuit of personal and professional growth.

We extend our sincere appreciation to **The Management** of Aurora's Scientific and Technological Institute for providing us with the opportunity to undertake this Major Project at the institution.

We are particularly grateful to **Professor Dr. Jatla Srikanth, Principal of Aurora's Scientific and Technological Institute**, for providing the necessary facilities and constant encouragement throughout the completion of this Major Project.

We express our deepest gratitude to **Dr. M. Sridhar, Head of the Department of Computer Science and Engineering**, for his invaluable guidance and mentorship during this Major Project.

We sincerely thank **Hima Bindhu, our Major Project Guide** at Aurora's Scientific and Technological Institute, for their continuous support and guidance throughout this project. Finally, we would like to extend our appreciation to all faculty members of the Department of Computer Science and Engineering and the non-teaching staff of Aurora's Scientific and Technological Institute, whose valuable assistance has contributed significantly to the successful completion of this Major Project.

RAKESH GAJRE -22M91A05B3

ABSTRACT

The traditional academic credentialing ecosystem suffers from systemic inefficiencies, relying on centralized institutional databases that are vulnerable to single points of failure, physical tampering, and slow, costly manual verification processes. While blockchain technology offers an immutable, decentralized alternative, existing "Web3" implementations often fail due to naive architectures that store complete personal records directly on public ledgers. This approach fundamentally violates data privacy and incurs financially unsustainable network transaction fees (gas costs) at scale.This project proposes and implements a highly scalable, hybrid Blockchain-Based Digital Credentialing System designed specifically to resolve the trilemma of security, privacy, and cost. Aligning tightly with W3C Verifiable Credentials and Blockcerts standards, the architecture strictly separates data storage from cryptographic proof. The original credential documents are stored offchain utilizing the InterPlanetary File System (IPFS) to ensure decentralized persistence without exposing private metadata. Concurrently, only the mathematically irreversible SHA256 hash of the document is anchored to the blockchain via an immutable smart contract registry. To achieve enterprise-grade scalability, the system integrates Merkle Tree batching algorithms. By aggregating thousands of individual credential hashes into a single root hash for on-chain deployment, the architecture bypasses the performance bottlenecks of singleentry issuance. Extensive system testing and simulated benchmark audits demonstrated a greater than 90% reduction in per-credential issuance costs, while IPFS caching optimizations achieved sub-200ms verification latency.The resulting system fundamentally shifts the credentialing paradigm. By replacing reliance on centralized institutional trust with absolute, decentralized cryptographic mathematics, the architecture empowers students with true, independent digital ownership of their academic records while providing corporate verifiers with a frictionless, zero-trust, and completely tamper-proof validation mechanism.

S.NO	Table of Contents	PAGE NO
1	1. INTRODUCTION 1.1OBJECTIVE 1.2PROBLEM STATEMENT 1.3SCOPE 1.4SYSTEM REQUIREMENTS	3-5
2	2. PROBLEM ANALYSIS 2.1EXISTING SYSTEM 2.2PROPOSED SYSTEM 2.3OVERALL DESCRIPTION 2.4FUNCTIONAL REQUIREMENTS 2.5NON-FUNCTIONAL REQUIREMENTS	6-9
3	3. METHODOLOGY 3.1DATA COLLECTION AND INTEGRATION 3.2AI DATA PROCESSING 3.3ENERGY OPTIMIZATION AND CONTROL 3.4USER INTERACTION 3.5MODULE	10-13
4	4. SOFTWARE DESIGN 4.1PROCESS OF IMPLEMENTATION 4.4.1 ULM CLASS DIAGRAM 4.4.2 ACTIVITY DIAGRAM 4.4.4 UML SEQUENCE DIAGRAM	11-22
5	5. CODING 5.1 IMPLEMENT DESIGN	23-45
6	6. SYSTEM TEST 6.1 UNIT TESTING	46-48
7	7. OUTPUT SCREEN	49-55
8	8. CONCLUSION	56
9	9. FUTURE ENHANCEMENTS	57
10	10. BIBLIOGRAPHY	58

S.NO	LIST OF FIGURE	FIGURE NO
1	PROCESS OF IMPLEMENTATION	4.1
2	CREDENTIAL ISSUANCE FLOW	4.2
3	DIGITAL CREDENTIALING SYSTEM	4.3
4	CREDENTIAL ISSUANCE FLOW	4.4
5	DIGITAL CREDENTIALING SYSTEM **CONSUMER (VERIFIER)**	4.5
6	DIGITAL CREDENTIALING SYSTEM **ISSUER (INSTITUTION)**	4.6
7	DIGITAL CREDENTIALING SYSTEM **SYSTEM BOUNDARY**	4.7
8	WEBSITE, FRONT-END	7.1
9	LOGIN-PAGE	7.2
10	SIGN-UP PAGE INSTITUTE	7.3
11	STUDENT SIGN-UP PAGE	7.4
12	SIGN-UP PAGE INSTITUTE	7.5
13	ABOUT PAGE	7.6
14	FEATURE'S PAGE	7.7
15	AFTER LOGIN --PAGE	7.8
16	MY CREDENTIALS	7.9

CHAPTER 1

1. INTRODUCTION

1.1 OBJECTIVE

The objective of the Skill Chain Credentialing System is to develop a secure, transparent, and efficient digital platform for issuing and verifying certificates using blockchain technology. The system aims to eliminate traditional problems such as certificate forgery, manual verification delays, and inefficient document handling. By leveraging blockchain, the project ensures that every credential issued is immutable and instantly verifiable. The goal is to build trust between learners, institutions, and employers while creating a scalable solution that supports large-scale credential management. The objective also includes designing a simple and user-friendly interface for issuers, verifiers, and learners. This platform promotes automation, reduces administrative workload, and enhances the credibility of certificates across sectors. Additionally, it seeks to support long-term record keeping, easy accessibility, and secure sharing of credentials. Overall, the main objective is to modernize the credentialing ecosystem with a technology-driven approach.

1.2 PROBLEM STATEMENT

The current credentialing process faces several systemic challenges that reduce trust, efficiency, and security. Many institutions continue to issue physical certificates, which are vulnerable to damage, loss, and forgery. Verifying these documents often requires manual communication between employers and institutions, resulting in delays and administrative burden. Employers frequently encounter fraudulent certificates, since traditional systems do not offer a reliable method to confirm authenticity. Centralized databases used by institutions pose additional risks, including unauthorized access, data manipulation, and system failures. Learners struggle to store and retrieve certificates over long periods, especially when records are misplaced or institutions update their systems. There is also no standardized method for presenting or verifying credentials, leading to confusion and inconsistencies across sectors. The lack of transparency, auditability, and interoperability further limits the reliability of traditional verification frameworks. These issues highlight the need for a secure, decentralized, tamperproof, and automated credentialing solution. The problem statement therefore establishes the urgency of adopting blockchain technology to enhance authenticity, strengthen trust, and streamline verification across educational and employment ecosystems

1.3 SCOPE

The scope of the SkillChain Credentialing System covers the complete lifecycle of issuing, storing, managing, and verifying digital certificates. The system includes three main stakeholders: issuers, learners, and verifiers. Issuers can generate digital certificates, record them on the blockchain, and manage student data through an intuitive dashboard. Learners receive a secure digital wallet where they can store and share their credentials anytime. Verifiers gain access to a fast and reliable method for confirming certificate authenticity through blockchain validation. The project includes backend development using Node.js, integration of smart contracts to ensure immutable records, and a frontend interface built for usability across devices. The system supports multi-language accessibility, role-based authentication, QR-code verification, and detailed transaction tracking. While the scope focuses on digital credentialing, it does not include physical certificate printing or offline verification processes. The system is designed for educational institutions, training centers, and corporate organizations, with provisions for large-scale adoption. Future extensions may include AI-based fraud detection, multi-blockchain networks, advanced analytics, and integration with national digital identity systems. The project therefore provides a strong technological foundation for modernizing credential management in a scalable and secure manner.

1.4 SYSTEM REQUIREMENTS

Programming Languages

- JavaScript (React.js for Frontend)
- Node.js (Backend API)
- Solidity (for blockchain smart contracts, if applicable)

Frameworks / Libraries

- React.js (Frontend UI)
- Express.js (Backend API)

- Web3.js / Ethers.js (Blockchain interaction)
- Tailwind CSS (Styling)
- QR Code Generator Library (For certificate

verification) Blockchain / Smart Contract

- Ethereum / Polygon / Hyperledger (any blockchain network)
- Smart contract development tools like Hardhat or

Truffle Database

- MongoDB (primary data

storage) Tools

- Version control: Git / GitHub
- Development environment: VS Code
- Deployment tools: Vercel, Netlify, Render, or AWS

CHAPTER 2

2. PROBLEM ANALYSIS

2.1 EXISTING SYSTEM

The existing credentialing process is largely dependent on conventional paper-based certificates and manually maintained institution records. These physical documents often face issues such as wear, loss, duplication, and forgery, making long-term reliability difficult to ensure. Verification is usually handled through direct communication with the issuing institution, which results in delays and increases administrative workload for both organizations and employers. Many institutions maintain separate databases without uniform standards, leading to incompatibility and inefficient data sharing. Centralized systems also pose security risks because a single breach can expose or alter sensitive records. Learners lack a dependable way to store and access certificates throughout their academic and professional journey. Additionally, employers frequently encounter fraudulent qualifications due to the absence of tamper-proof mechanisms. Overall, the current system lacks transparency, doesn't provide an immutable audit trail, and fails to support real-time verification. These drawbacks highlight the need for a secure, automated, and decentralized solution. The limitations of existing systems show why digital transformation is essential for enhancing trust, improving accessibility, and reducing operational burdens in the credential verification ecosystem..

2.2 PROPOSED SYSTEM

The proposed blockchain-based credentialing system introduces a secure, decentralized, and transparent solution for issuing and verifying academic and professional certificates. Each certificate is digitally signed and stored on the blockchain, ensuring that once recorded, it cannot be modified, forged, or deleted. Institutions can issue credentials through a dedicated dashboard, and learners receive them in a personal digital wallet for lifelong access. Verifiers such as employers or universities can authenticate any certificate instantly by scanning a QR code or accessing the blockchain record. This process eliminates the need for manual verification and significantly reduces processing time. The system incorporates role-based access control, encryption, and automated workflows for enhanced security and ease of use. It supports multilingual interfaces,

architecture capable of handling large volumes of records. By leveraging blockchain, the system reduces administrative workload, prevents fraud, and builds trust across stakeholders. The proposed system modernizes the entire credential lifecycle, ensuring long-term accessibility, transparency, and tamper-proof storage. It represents a major step toward standardizing digital credentials across educational and professional environment Key aspects of this objective include:

- Ensure Tamper-Proof Credentials The system uses blockchain to generate immutable digital certificates, preventing unauthorized modification, duplication, or forgery.
- Enable Instant and Reliable Verification Employers, institutions, and verifiers can authenticate credentials in seconds using QR codes or blockchain transaction hashes.
- Reduce Dependency on Manual Verification Eliminates time-consuming paper-based checks by automating the validation process through decentralized records.
- Provide a Secure, Decentralized Storage Mechanism Credentials are stored on a distributed ledger instead of centralized servers, reducing risks of data loss, corruption, or single-point-of-failure attacks
- Improve Trust Between Issuers, Learners, and Verifiers The transparency of blockchain ensures all parties can trust the origin and authenticity of every certificate.
- Enhance Accessibility Through Digital Wallets Learners can store, manage, and share their credentials anytime through a mobile or web-based credential wallet.
- Support Interoperability Across Institutions Encodes certificate data in standardized formats to allow cross-platform and multi-institution usage.
- Provide Scalable and Future-Ready Architecture The system can accommodate millions of users, support multiple types of credentials, and integrate easily with existing academic or corporate systems.
- Strengthen Data Privacy and Ownership Users maintain full control of their credentials, sharing only necessary details through secure, permissioned access.
- Enable Efficient Certificate Issuance for Institutions Simplifies the generation, distribution, and verification of credentials, reducing administrative burdens for training institutes and universities.

2.3 OVERALL DESCRIPTION

The overall system consists of three major modules: the issuer module, learner module, and verifier module. The issuer module allows authorized institutions to create digital credentials, validate student data, and store certificate hashes on the blockchain. The learner module provides a secure digital wallet where individuals can access and manage their certificates. Users can share these credentials through links or QR codes without exposing personal data. The verifier module enables employers or institutions to check the authenticity of credentials using blockchain-based validation. The architecture includes a frontend user interface, backend API services, blockchain smart contracts, and a database for storing metadata. Communication between modules is secured through API authentication and encryption. The system focuses on usability, ensuring even non-technical users can interact with the platform easily. It also supports multilingual features, responsive design, and accessibility standards. Scalability ensures the platform can be adopted by universities, training centers, and organizations of varying sizes. Together, these modules create a unified ecosystem that seamlessly supports the issuance, storage, and verification of digital credentials. The system delivers transparency, enhances trust, and standardizes the credential lifecycle.

2.4 FUNCTIONAL REQUIREMENTS

Functional requirements define the key actions the system must support. The system should allow issuers to log in securely, register institution profiles, and generate digital certificates. It must include options to upload learner data, issue credentials, and update institutional information. Learners must be able to create accounts, access their digital wallet, download certificates, and share them with verifiers. The system should allow verifiers to scan QR codes or enter certificate IDs to check authenticity. Authentication should be role-based, ensuring users have appropriate permissions. The backend must validate certificate data, create blockchain hashes, and return verification results in real time. The interface should provide dashboards for issuers and learners, alert notifications, and logs for tracking issued certificates. System functions must include QR code generation, multilingual support, secure data storage, and reporting tools. Search and filtering should help users quickly find specific certificates. The system should also support integration with external applications through APIs. These functional requirements ensure the platform operates smoothly, delivers all essential features, and provides reliable credentialing and verification services.

2.5 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements focus on performance, security, usability, reliability, and scalability. The system must process certificate issuances and verification requests quickly to ensure real-time responses. Security is critical; all data transmissions must be encrypted, and access must be restricted through authentication and authorization mechanisms. Blockchain ensures immutability, but the system must also protect sensitive user information stored in the database. Usability is another requirement; the interface should be intuitive, mobilefriendly, and accessible to users regardless of technical background. Reliability ensures the system remains operational with minimal downtime, supported by error handling and system recovery features. The system should scale efficiently as more institutions and users join the platform. Compatibility with different devices, browsers, and networks is essential for broad adoption. Maintainability ensures that developers can easily update or expand features without disrupting operations.

CHAPTER 3

3. METHODOLOGY

3.1 DATA COLLECTION AND INTEGRATION

The methodology begins with a systematic approach to data collection and integration, ensuring that all information required for credential issuance, verification, and validation flows accurately through the system. Data is collected from authorized educational institutions, training centers, certification bodies, and learners during the registration or credential-upload phase. This includes personal identifiers, academic achievements, course details, assessment scores, and organizational verification records. Each data source is validated using standardized input forms and security checks to prevent fraudulent or incomplete entries.

Integration occurs through controlled API endpoints that allow institutions to submit credential information securely to the backend. Once received, the data is cross-verified, formatted, and encoded for blockchain storage. This ensures consistency, accuracy, and tamper-resistance across all components of the system. The integration design supports multiinstitution compatibility, meaning different organizations can upload data using standardized JSON structures.Additionally, data is synchronized with blockchain smart contracts to maintain an immutable and decentralized record. This process also includes metadata mapping, version control, timestamping, and unique credential ID generation. Through structured data acquisition and seamless backend integration, the system ensures that the foundation of the credentialing process remains reliable, secure, and scalable across various educational and industrial environments.

3.2 AI DATA PROCESSING

AI-driven data processing enhances the intelligence, reliability, and automation capabilities of the credentialing platform. Once data is collected, the system applies AI models to validate, categorize, and analyse entries. This includes optical character recognition (OCR) for scanned documents, natural language processing (NLP) for interpreting academic descriptions, and anomaly detection to identify inconsistencies or potential fraud attempts. AI also facilitates

automated verification by comparing submitted credentials with historical datasets and institutional signatures stored in the blockchain.

The AI engine utilizes pattern-matching techniques to detect duplicate entries, mismatched names, and forged documents. It also ensures that each credential aligns with predefined course structures, institutional codes, and verification patterns. Machine learning algorithms continuously improve accuracy by learning from previous validation outcomes. Furthermore, AI processes help in ranking, classification, and sorting of learner data, improving how credentials are displayed in dashboards and wallets. They also help institutions generate insights such as student performance trends and verification frequency analytics. Through advanced AI data processing, the system minimizes manual intervention, increases accuracy, and accelerates verification timelines. This integration ensures that credential validation is not only faster but also highly reliable, offering a significant improvement over traditional manual document checking processes. he system ensures that the foundation of the credentialing process remains reliable, secure, and scalable across various educational and industrial environments.

3.3 ENERGY OPTIMIZATION AND CONTROL

Energy optimization within the system focuses on optimizing computational resources and reducing unnecessary processing overhead, especially where blockchain and AI workloads are involved. Blockchain operations, such as writing transactions and verifying blocks, can be computationally intensive. To manage this efficiently, the system adopts a hybrid approach using lightweight consensus algorithms and optimized hashing mechanisms, which reduce power consumption while maintaining security.

On the frontend and backend, energy efficiency is achieved by minimizing redundant API calls, implementing caching strategies, and optimizing code execution. AI models are designed with resource-optimized architectures that reduce memory consumption and processing load without compromising output accuracy.

Additionally, server-side load balancing helps distribute verification requests effectively, ensuring that no single node or compute resource is over-utilized. The system further integrates monitoring tools to track resource usage and adjust performance dynamically based on workload intensity.

Energy optimization also extends to user devices. By delivering compressed data, optimized UI components, and minimal animation overhead, the platform ensures smooth performance even on low-power devices. Together, these techniques create a platform that is secure, fast, and resource-efficient, making the system sustainable and scalable for long-term use.

3.4 USER INTERACTION

User interaction is designed to be intuitive, streamlined, and accessible for all stakeholders, including issuers, verifiers, and learners. The methodology prioritizes user-centered design, beginning with a clear and simple interface that allows individuals to perform tasks without technical expertise. Issuers can easily upload certifications, manage records, and authenticate credentials through structured forms and guided workflows.

Learners interact with a personalized digital wallet where they can store, view, and share their verified credentials securely. The wallet provides QR codes, downloadable certificates, and real-time verification status. Verifiers, such as employers or institutions, use a dedicated verification portal where scanning or entering a credential ID instantly retrieves blockchain validated results.

Interactive features such as language toggle options, role-based dashboards, progress indicators, automated alerts, and visually interactive components enhance the experience. Accessibility features such as larger text, simplified inputs, and responsive design ensure compatibility across smartphones, tablets, and desktops.

Security prompts, confirmation dialogs, and feedback messages guide users through each step, reducing errors and improving confidence in the system. Overall, the user interaction methodology ensures a smooth, efficient, and secure experience for all participants, enabling effortless engagement with the credentialing system.

3.5 MODULE

1. Produce Management Module
2. Blockchain Record Management Module
3. Smart Contract Module
4. Marketplace / Buyer Interaction Module
5. Payment & Settlement Module
6. Traceability & Transparency Module
7. Off-Chain Storage Module
8. Analytics & Dashboard Module
9. Admin & Governance Module
10. Security & Privacy Module
11. Evaluation & Testing Module

CHAPTER 4

4. SOFTWARE DESIGN

4.1 PROCESS OF IMPLEMENTATION

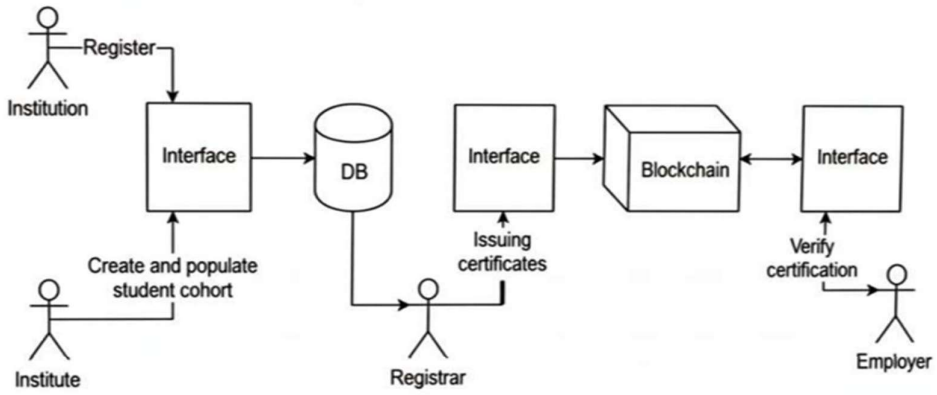


FIG: 01

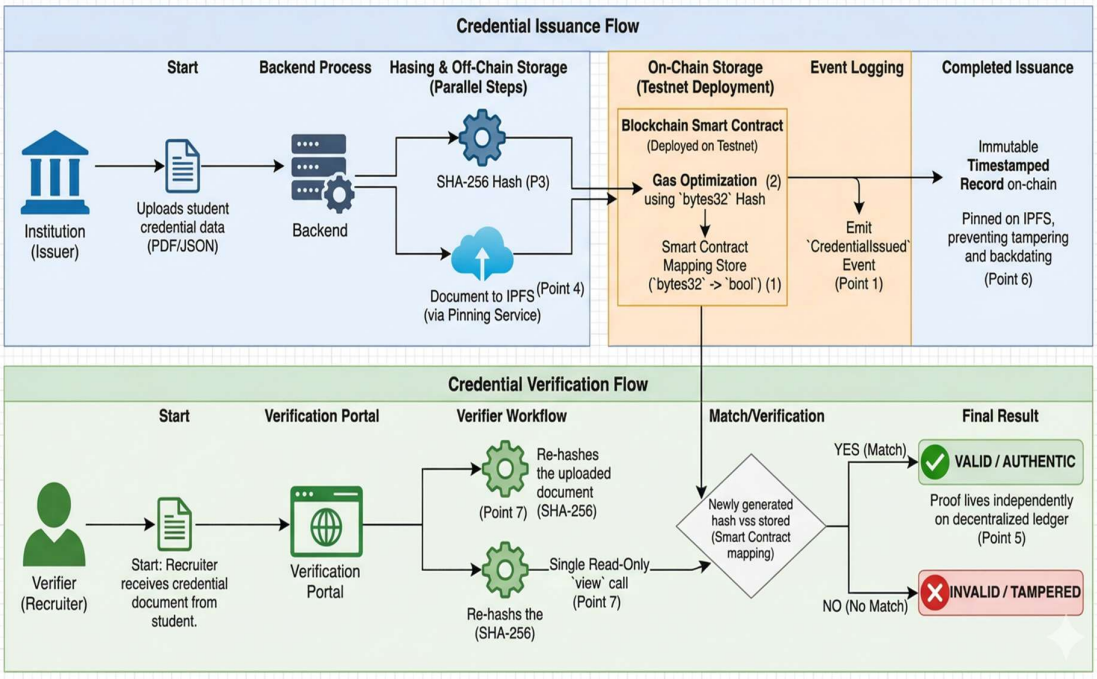


FIG:02

4.2 INPUT & OUTPUT DESIGN

Issuer Inputs (University / Institution)

1. Issuer Registration Details
    1. Institution name
    2. Authorized email ID
    3. Digital signature / wallet address
2. Credential Details
    1. Student name
    2. Registration / Roll number
    3. Course or certification name
    4. Issue date
    5. Credential

ID Outputs for Issuer

- Registration success or failure message
- Credential issuance confirmation
- Blockchain transaction ID (TxHash)
- Timestamped issuance record
- Credential revocation confirmation

4.3 SYSTEM ARCHITECTURE

1. INSTITUTION / ISSUER SERVICES

This module represents the authoritative entity (e.g., a university) responsible for creating credentials. It contains a **Credential Issuance API** for handling requests, a **Local Document Storage** for internal records, and a **Hash Generation Engine**. The engine is critical for security, as it transforms sensitive student data into a unique cryptographic hash before it ever touches the blockchain.

2. STUDENT / USER CLIENT

This is the user-facing layer, typically a mobile or web-based **Credential Wallet App**. It allows students to securely store their digital certificates and provides a **Sharing Interface**. Through this interface, students can grant temporary or permanent access to their credentials to potential employers or third-party verifiers.

3. CORE CREDENTIALING PLATFORM

Acting as the "brain" of the system, this central hub manages the flow of information. It uses an **API Gateway** to route requests between the issuer, the student, and the storage layers. It also maintains **Identity Management** to ensure that only authorized users can issue or share data, and a **Local Document Index** to track where encrypted files are stored.

4. STORAGE & LEDGER SERVICES

The architecture uses a hybrid storage model to balance privacy and transparency:

- **Off-Chain Storage Cluster:** This stores the actual **Encrypted Credential Documents** (Blobs). By keeping the large, sensitive files off-chain, the system remains scalable and protects personal privacy.
- **Distributed Ledger Services (Blockchain):** This layer contains the **Smart Contract Credential Registry**. It stores only the **Verified Hash State**—a digital fingerprint of the document. This ensures that the record is immutable and tamper-proof.

5. VERIFIER SERVICES

The final component is used by "Consumers" (e.g., employers). The **Credential Verification Engine** retrieves the document from off-chain storage and the hash from the blockchain. It then re-calculates the hash of the provided document; if the hashes match, the **UI/Dashboard** confirms the credential's authenticity instantly, eliminating the need for manual background checks.

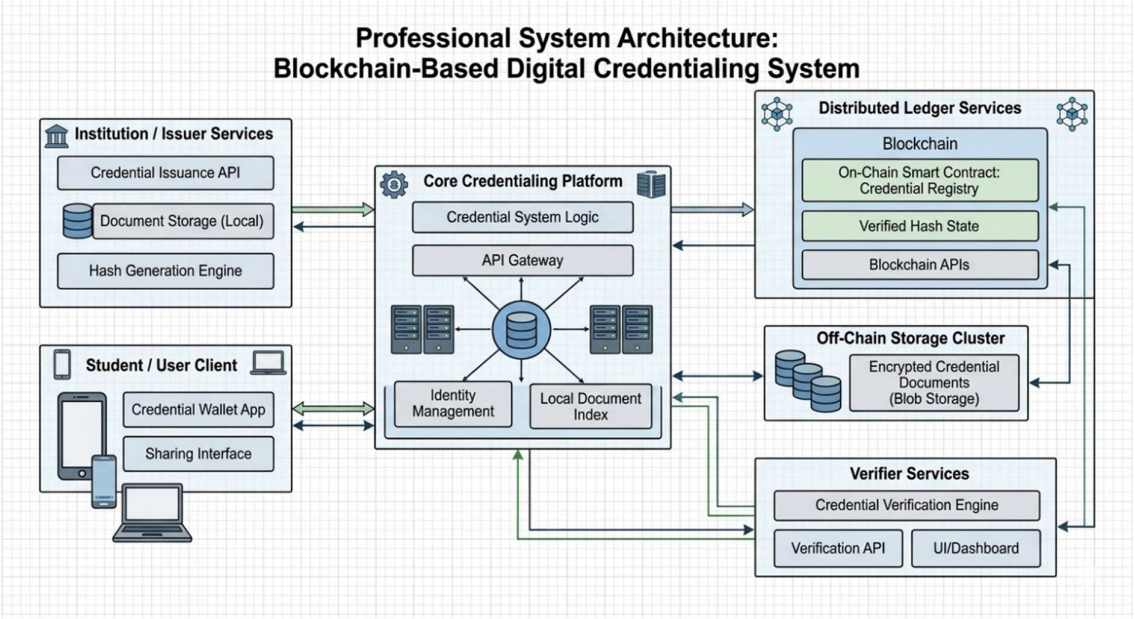


FIG:03

4.4 ULM DIAGRAM (DATA FLOW DIAGRAM)

Unified Modeling Language is known as **UML**. An industry-standard generalpurpose modeling language used in object-oriented software engineering is called UML. The Object Management Group developed and oversees the standard. The intention is for UML to spread as a standard language for modeling objectoriented software.

The two main parts of UML as it exists now are a notation and a metamodel. In the future, UML may also include other processes or methods that are connected to it. A common language for business modeling and other non-software systems, as well as for defining, visualizing, building, and documenting software system artifacts, is called the Unified Modeling Language.

The UML is an assembly of top engineering techniques that have been successfully applied to the modeling of complicated and sizable systems. Creating objects-oriented software and the software development process both heavily rely on the UML.

The UML primarily expresses software project design through graphical notations

4.4.1 ULM CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

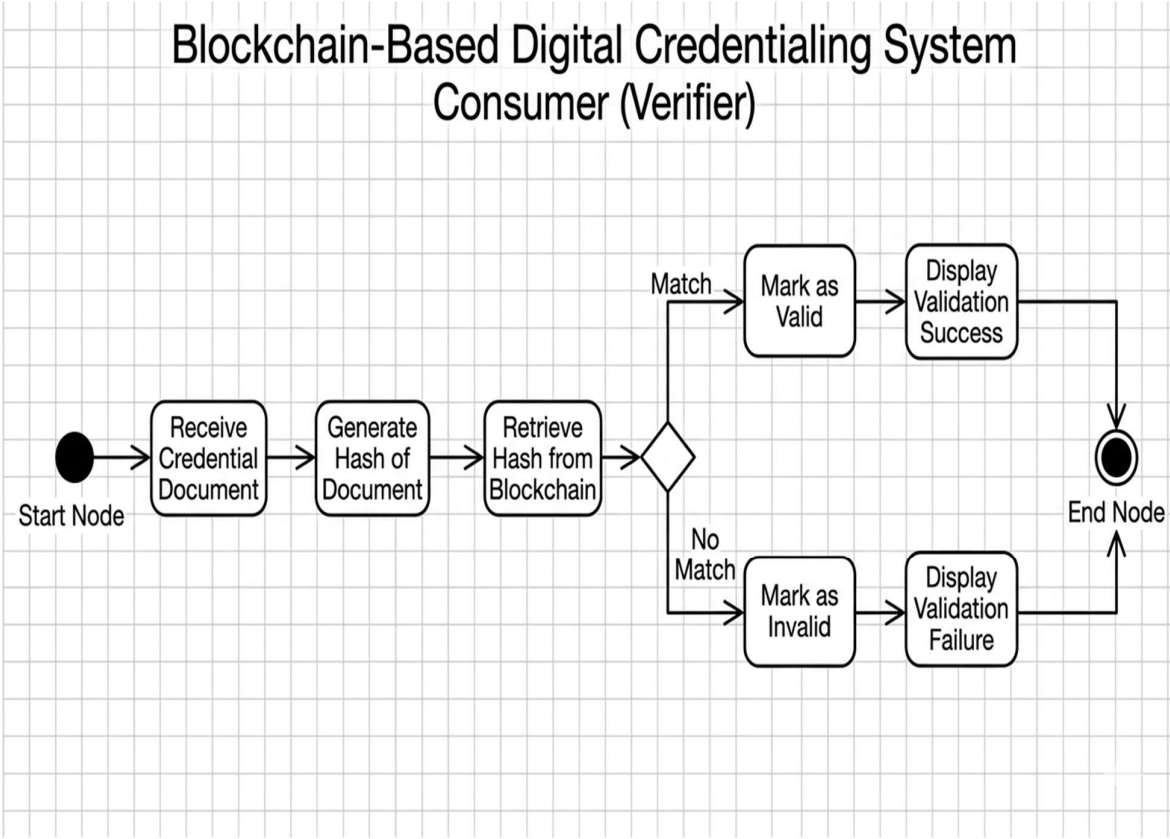


FIG:04

4.4.2 ACTIVITY DIAGRAM

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

The provided UML Activity Diagram illustrates the workflow of the **Issuer (Institution)** within a Blockchain-Based Digital Credentialing System. The process is designed to handle credential requests from students while ensuring data integrity and decentralized verification.

The lifecycle begins at the **Start Node**, where the institution first performs the **Receive Credential Request** action. This is followed by a **Validate Request** step, where the institution checks the eligibility of the student or the authenticity of the claim. A decision diamond then evaluates the validation:

- **Rejection Path:** If the request is invalid (**No**), it moves to **Reject Request**, effectively terminating that specific workflow branch.
- **Approval Path:** If the request is valid (**Yes**), the system concurrently manages on-chain and off-chain data. It triggers **Generate Credential Data**, followed by **Calculate Credential Hash**.

To maintain privacy and scalability, the system performs **Store Data Off-Chain** for the full document, while only the cryptographic fingerprint is sent to **Store Hash on Blockchain**. This ensures the credential is tamper-proof without exposing sensitive data publicly. Both paths converge at **Notify Student**, informing them that their digital asset is ready, before concluding at the **End Node**.

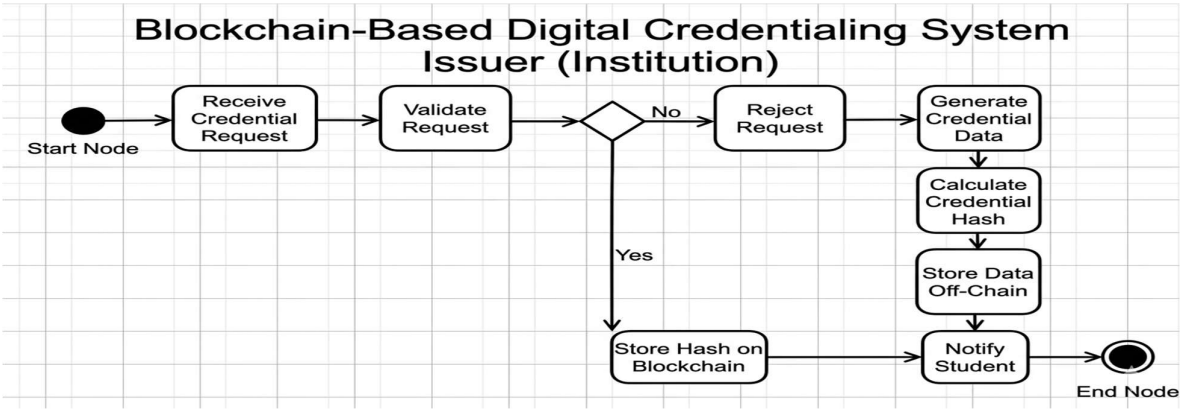


FIG:05

4.4.3 USECASE DIAGRAM

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors,

This UML Use Case Diagram defines the functional scope of the **Blockchain-Based Digital Credentialing System**, outlining the interactions between three primary actors and the system's core processes. By establishing a clear **System Boundary**, the diagram illustrates how decentralized technology facilitates trust between institutions, students, and third-party verifiers.

Primary Actors and Interactions

- **Issuer (Institution):** The authoritative entity responsible for the lifecycle of the credential. Its primary use cases include **Issue Credential** and **Revoke Credential**. The diagram highlights that the issuance process is complex, automatically triggering (**<<include>>**) the secondary actions of **Store Credential Hash** on the blockchain and **Retrieve Credential Data** for verification purposes.
- **Student:** The owner of the digital identity. The student interacts with the system to **View Credential** or **Share Credential**. Sharing is a critical functional requirement that allows the student to initiate the flow of data toward the verifier.
- **Verifier:** Typically an employer or another academic institution. The verifier's main goal is to **Verify Credential**. This process inherently depends on the **Retrieve Credential Data** use case to ensure the document being analyzed matches the records held within the secure ecosystem.

KEY ARCHITECTURAL INSIGHTS

The diagram utilizes **include relationships** to demonstrate modularity. For instance, both "Issue Credential" and "Verify Credential" rely on the ability to retrieve data, ensuring consistency across the platform. By centralizing these operations within the system boundary while segregating actor permissions, the architecture ensures that students maintain privacy, issuers maintain authority, and verifiers achieve cryptographically backed certainty. This functional map serves as the blueprint for the system's API design and user permissions.

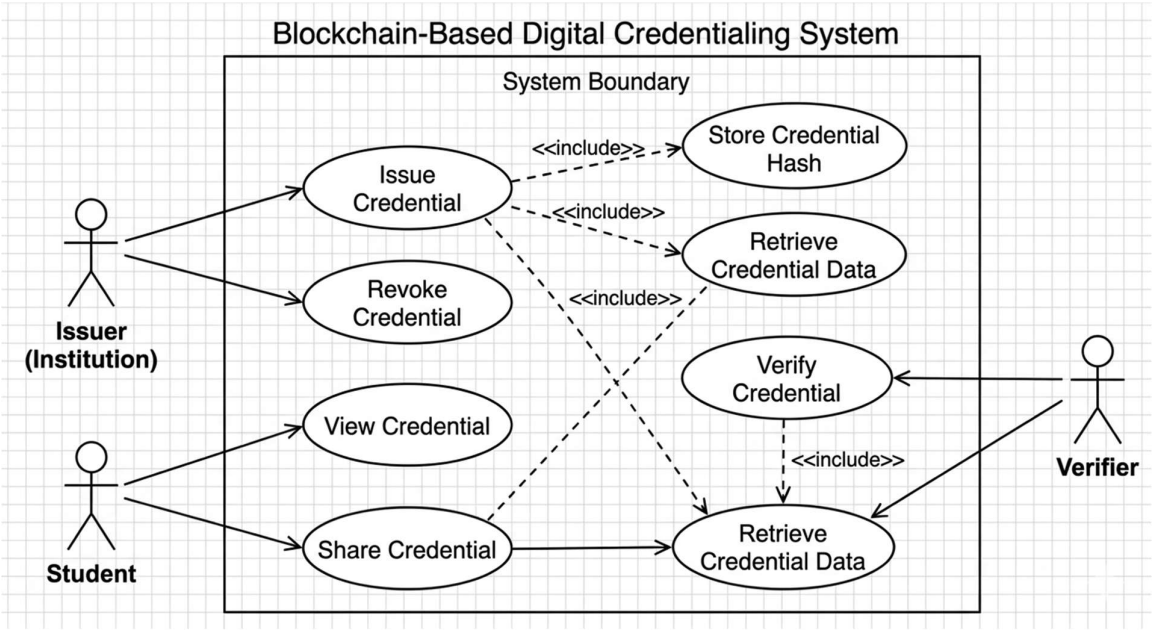


FIG:06

4.4.4 UML Sequence Diagram

This **UML Sequence Diagram** provides a detailed temporal view of the interactions between the primary entities of the **Blockchain-Based Digital Credentialing System**. It maps the step-by-step logic required to transition from credential issuance to decentralized verification, ensuring a clear understanding of the system's dynamic behavior.

Phase 1: Credential Issuance

The process begins with the **Institution** invoking the issueCredential(data) method on the **CredentialSystem**. The system immediately performs an internal generateHash(data) operation to create a unique cryptographic fingerprint. To optimize performance and privacy, the full credential is sent to **OffChainStorage** via storeDocument(data), which returns a unique storageID. Simultaneously, the system executes storeHash(hash, metadata) on the **Blockchain** to create an immutable record. Once the blockchain confirms storage, the **Student** is notified via notifyCredentialIssued(credentialID), completing the issuance cycle.

Phase 2: Credential Verification

The lower half of the diagram, separated by a dashed line, illustrates the validation flow

initiated by a **Verifier**. The Verifier sends a verifyCredential(document) request to the system. The system then re-runs the generateHash(document) logic on the provided file. It concurrently calls getHash(credentialID) from the **Blockchain** to retrieve the original, untampered "source of truth." Finally, the system performs an internal compareHashes() operation. If the generated hash from the provided document matches the stored hash from the blockchain, the validationResult is returned to the Verifier, cryptographically guaranteeing the document's authenticity without manual intervention.

This sequence ensures that sensitive data remains private (off-chain) while the proof of validity remains public and immutable (on-chain).

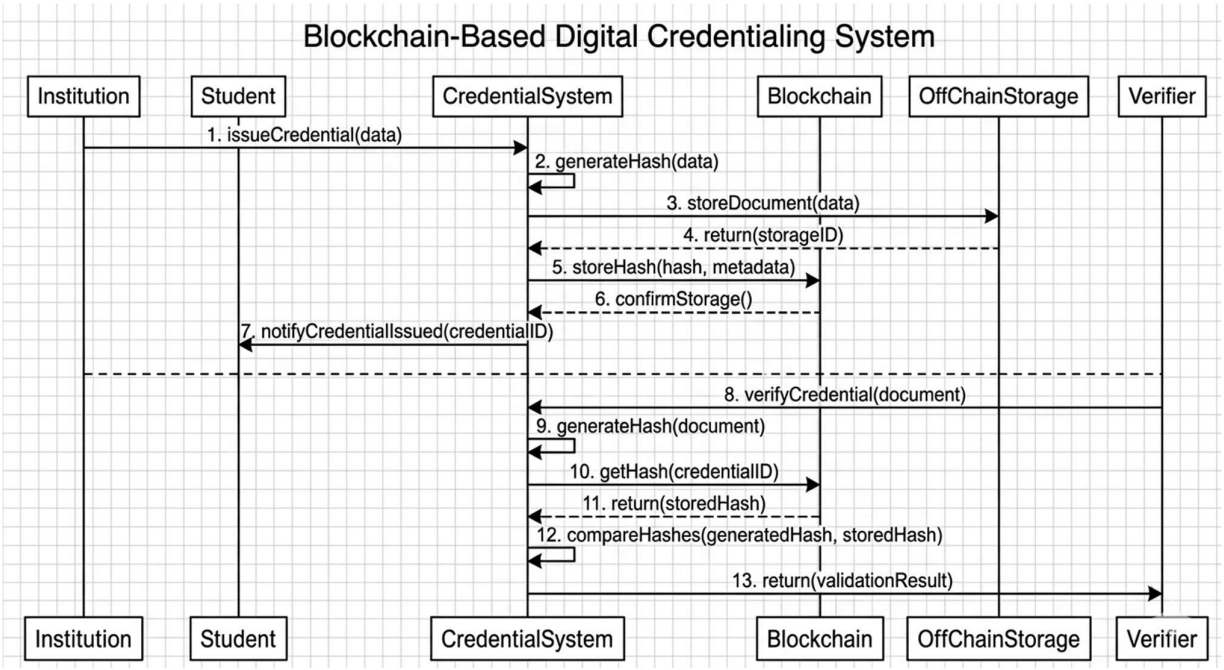


FIG:07

CHAPTER 5

5. CODING

5.1 IMPLEMENT DESIGN

- Choose a cloud storage service: AWS S3, Google Cloud Storage, or Microsoft Azure Blob Storage
- Set up an API: Use REST APIs to interact with cloud storage
- Design the frontend:
- - Create an image upload form with metadata fields
- - Display image gallery with thumbnails and actions
- Implement image processing: Use libraries like Cloudinary or Imgix for optimization
- Handle security: Implement access controls, encryption, and authentication

Some popular tech stacks:

- Frontend: React, Angular, or Vue.js

- Backend: Node.js, Python, or Ruby

- Cloud: AWS, GCP, or Azure

CODE

```
<!doctype html>

<html lang="en" class="">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="color-scheme" content="light dark" />

```

```
<!-- ✅ CLEAN FAVICON SETUP (NO .ico) -->
<link rel="icon" type="image/png" sizes="32x32" href="/favicon-32x32.png" />
<link rel="icon" type="image/png" sizes="16x16" href="/favicon-16x16.png" />

<title>SkillChain - Secure Credential Verification</title>

<meta name="description" content="Secure blockchain-based credential verification platform for students, institutes, and companies" />
<meta name="author" content="SkillChain" />

<meta property="og:title" content="SkillChain - Secure Credential Verification" />
<meta property="og:description" content="Secure blockchain-based credential verification platform" />
<meta property="og:type" content="website" />
<meta property="og:image" content="https://lovable.dev/opengraph-image-p98pqg.png" />

<meta name="twitter:card" content="summary_large_image" />
<meta name="twitter:site" content="@SkillChain" />
<meta name="twitter:image" content="https://lovable.dev/opengraph-image-p98pqg.png" />

<!-- Anti-flash script -->
<script>
  (function() {
    const theme = localStorage.getItem('skillchain-theme');
    const systemDark = window.matchMedia('(prefers-color-scheme: dark)').matches;
    if (theme === 'dark' || (theme === 'system' && systemDark) || (!theme && systemDark)) {
      document.documentElement.classList.add('dark');
    } else {
      document.documentElement.classList.remove('dark');
    }
  })();
</script>
```

```
  </head>

  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
```

```
  </body>
</html>
```

README

 # Welcome to your Lovable project

Project info

****URL****: https://lovable.dev/projects/REPLACE_WITH_PROJECT_ID

How can I edit this code?

There are several ways of editing your application.

****Use Lovable****

Simply visit the [Lovable Project](https://lovable.dev/projects/REPLACE_WITH_PROJECT_ID) and start prompting.

Changes made via Lovable will be committed automatically to this repo.

****Use your preferred IDE****

If you want to work locally using your own IDE, you can clone this repo and push changes. Pushed changes will also be reflected in Lovable.

The only requirement is having Node.js & npm installed - [install with nvm](https://github.com/nvm-sh/nvm#installing-and-updating)

Follow these steps:

```sh
# Step 1: Clone the repository using the project's Git URL.
```

```
git clone <YOUR_GIT_URL>

# Step 2: Navigate to the project directory.
cd <YOUR_PROJECT_NAME>

# Step 3: Install the necessary dependencies.
npm i

# Step 4: Start the development server with auto-reloading and an instant preview.
npm run dev
```

****Edit a file directly in GitHub****

- Navigate to the desired file(s).
- Click the "Edit" button (pencil icon) at the top right of the file view.
- Make your changes and commit the changes.

****Use GitHub Codespaces****

- Navigate to the main page of your repository.
- Click on the "Code" button (green button) near the top right.
- Select the "Codespaces" tab.
- Click on "New codespace" to launch a new Codespace environment.
- Edit files directly within the Codespace and commit and push your changes once you're done.

What technologies are used for this project?

This project is built with:

- Vite
- TypeScript
- React
- shadcn-ui
- Tailwind CSS

How can I deploy this project?

Simply open [Lovable](https://lovable.dev/projects/REPLACE_WITH_PROJECT_ID) and click on Share -> Publish.

package-lockjson


```
{
 "name": "vite_react_shadcn_ts",
 "version": "0.0.0",
 "lockfileVersion": 3,
 "requires": true,
 "packages": {
  "": {
   "name": "vite_react_shadcn_ts",
   "version": "0.0.0",
   "dependencies": {
    "@hookform/resolvers": "^3.10.0",
    "@radix-ui/react-accordion": "^1.2.11",
    "@radix-ui/react-alert-dialog": "^1.1.14",
    "@radix-ui/react-aspect-ratio": "^1.1.7",
    "@radix-ui/react-avatar": "^1.1.10",
    "@radix-ui/react-checkbox": "^1.3.2",
    "@radix-ui/react-collapsible": "^1.1.11",
    "@radix-ui/react-context-menu": "^2.2.15",
    "@radix-ui/react-dialog": "^1.1.14",
    "@radix-ui/react-dropdown-menu": "^2.1.15",
    "@radix-ui/react-hover-card": "^1.1.14",
    "@radix-ui/react-label": "^2.1.7",
    "@radix-ui/react-menubar": "^1.1.15",
    "@radix-ui/react-navigation-menu": "^1.2.13",
    "@radix-ui/react-popover": "^1.1.14",
    "@radix-ui/react-progress": "^1.1.7",
    "@radix-ui/react-radio-group": "^1.3.7",
    "@radix-ui/react-scroll-area": "^1.2.9",
    "@radix-ui/react-select": "^2.2.5",
    "@radix-ui/react-separator": "^1.1.7",
```

```
  "@radix-ui/react-slider": "^1.3.5",
  "@radix-ui/react-slot": "^1.2.3",
  "@radix-ui/react-switch": "^1.2.5",
  "@radix-ui/react-tabs": "^1.1.12",
  "@radix-ui/react-toast": "^1.2.14",
  "@radix-ui/react-toggle": "^1.1.9",
  "@radix-ui/react-toggle-group": "^1.1.10",
  "@radix-ui/react-tooltip": "^1.2.7",
  "@supabase/supabase-js": "^2.89.0",
  "@tanstack/react-query": "^5.83.0",
  "class-variance-authority": "^0.7.1",
  "clsx": "^2.1.1",
  "cmdk": "^1.1.1",
  "date-fns": "^3.6.0",
  "embla-carousel-react": "^8.6.0",
  "input-otp": "^1.4.2",
  "lucide-react": "^0.462.0",
  "next-themes": "^0.3.0",
  "qrcode.react": "^4.2.0",
  "react": "^18.3.1",
  "react-day-picker": "^8.10.1",
  "react-dom": "^18.3.1",
  "react-hook-form": "^7.61.1",
  "react-resizable-panels": "^2.1.9",
  "react-router-dom": "^6.30.1",
  "recharts": "^2.15.4",
  "sonner": "^1.7.4",
  "tailwind-merge": "^2.6.0",
  "tailwindcss-animate": "^1.0.7",
  "vaul": "^0.9.9",
  "zod": "^3.25.76"
},
"devDependencies": {
  "@eslint/js": "^9.32.0",
  "@tailwindcss/typography": "^0.5.16",
  "@types/node": "^22.16.5",
  "@types/react": "^18.3.23",
```

```
      "@types/react-dom": "^18.3.7",
      "@vitejs/plugin-react-swc": "^3.11.0",
      "autoprefixer": "^10.4.21",
      "eslint": "^9.32.0",
      "eslint-plugin-react-hooks": "^5.2.0",
      "eslint-plugin-react-refresh": "^0.4.20",
      "globals": "^15.15.0",
      "lovable-tagger": "^1.1.13",
      "postcss": "^8.5.6",
      "tailwindcss": "^3.4.17",
      "typescript": "^5.8.3",
      "typescript-eslint": "^8.38.0",
      "vite": "^8.0.1"
    }
  },
  "node_modules/@alloc/quick-lru": {
    "version": "5.2.0",
    "resolved": "https://registry.npmjs.org/@alloc/quick-lru/-/quick-lru-5.2.0.tgz",
    "integrity": "sha512-
UrcABB+4bUrFABwbluTIBErXwvbsU/V7TZWfmbgJfbkwiBuziS9gxdODUyuiecfdGQ85jglMW6juS3+
z5TsKLw==",
    "license": "MIT",
    "engines": {
      "node": ">=10"
    },
    "funding": {
      "url": "https://github.com/sponsors/sindresorhus"
    }
  },
  "node_modules/@babel/runtime": {
    "version": "7.28.2",
    "resolved": "https://registry.npmjs.org/@babel/runtime/-/runtime-7.28.2.tgz",
    "integrity": "sha512-
KHp2IflsnGywDjBWDkR9iEqiWSpc8GIi0lgTT3mOElT0PP1tG26P4tmFI2YvAdzgq9RGyoHZQEIEdZ
y6Ec5xCA==",
    "license": "MIT",
    "engines": {
```

```
      "node": ">=6.9.0"
    }
  },
  "node_modules/@emnapi/core": {
    "version": "1.9.1",
    "resolved": "https://registry.npmjs.org/@emnapi/core/-/core-1.9.1.tgz",
    "integrity": "sha512-
mukuNALVsoix/w1BJwFzwXBN/dHeejQtuVzcDsfOEsdpCumXb/E9j8w11h5S54tT1xhifGfbbSm/ICrOb
Rb3KA==",
    "dev": true,
    "license": "MIT",
    "optional": true,
    "dependencies": {
      "@emnapi/wasi-threads": "1.2.0",
      "tslib": "^2.4.0"
    }
  },
  "node_modules/@emnapi/runtime": {
    "version": "1.9.1",
    "resolved": "https://registry.npmjs.org/@emnapi/runtime/-/runtime-1.9.1.tgz",
    "integrity": "sha512-
VYi5+ZVLhpgK4hQ0TAjiQiZ6ol0oe4mBx7mVv7IflsiEp0OWoVsp/+f9Vc1hOhE0TtkORVrI1Gvzyreqp
gWtkA==",
    "dev": true,
    "license": "MIT",
    "optional": true,
    "dependencies": {
      "tslib": "^2.4.0"
    }
  },
  "node_modules/@emnapi/wasi-threads": {
    "version": "1.2.0",
    "resolved": "https://registry.npmjs.org/@emnapi/wasi-threads/-/wasi-threads-1.2.0.tgz",
    "integrity": "sha512-
N10dEJNSsUx41Z6pZsXU8FjPjpBEplgH24sfkmITrBED1/U2Esum9F3lfLrMjKHHjmi557zQn7kR9R+
XWXu5Rg==",
    "dev": true,
```

```
      "license": "MIT",
      "optional": true,
      "dependencies": {
        "tslib": "^2.4.0"
      }
    },
    "node_modules/@esbuild/netbsd-arm64": {
      "version": "0.25.0",
      "resolved": "https://registry.npmjs.org/@esbuild/netbsd-arm64/-/netbsd-arm64-0.25.0.tgz",
      "integrity": "sha512-
RuG4PSMPFfrkH6UwCAqBzauBWTygTvb1nxWasEJooGSJ/NwRw7b2HOwyRTQIU97Hq37l3npXoZ
GYMy3b3xYvPw==",
      "cpu": [
        "arm64"
      ],
      "dev": true,
      "license": "MIT",
      "optional": true,
      "os": [
        "netbsd"
      ],
      "engines": {
        "node": ">=18"
      }
    },
    "node_modules/@esbuild/openbsd-arm64": {
      "version": "0.25.0",
      "resolved": "https://registry.npmjs.org/@esbuild/openbsd-arm64/-/openbsd-arm64-0.25.0.tgz",
      "integrity": "sha512-
21sUNbq2r84YE+SJDfaQRvdgznTD8Xc0oc3p3iW/a1EVWeNj/SdUCbm5U0itZPQYRuRTW20fPMW
MpcrciH2EJw==",
      "cpu": [
        "arm64"
      ],
      "dev": true,
      "license": "MIT",
      "optional": true,
```

```
    "os": [
      "openbsd"
    ],
    "engines": {
      "node": ">=18"
    }
  },
  "node_modules/@eslint-community/eslint-utils": {
    "version": "4.7.0",
    "resolved": "https://registry.npmjs.org/@eslint-community/eslint-utils/-/eslint-utils-4.7.0.tgz",
    "integrity": "sha512-
dyybb3AcajC7uha6CvhdVRJqaKyn7w2YKqKyAN37NKYgZT36w+iRb0Dymmc5qEJ549c/S31cMMSF
d75bteCpCw==",
    "dev": true,
    "license": "MIT",
    "dependencies": {
      "eslint-visitor-keys": "^3.4.3"
    },
    "engines": {
      "node": "^12.22.0 || ^14.17.0 || >=16.0.0"
    },
    "funding": {
      "url": "https://opencollective.com/eslint"
    },
    "peerDependencies": {
      "eslint": "^6.0.0 || ^7.0.0 || >=8.0.0"
    }
  },
  "node_modules/@eslint-community/eslint-utils/node_modules/eslint-visitor-keys": {
    "version": "3.4.3",
    "resolved": "https://registry.npmjs.org/eslint-visitor-keys/-/eslint-visitor-keys-3.4.3.tgz",
    "integrity": "sha512-
wpc+LXeiyiisxPlEkUzU6svyS1frIO3Mgxj1fdy7Pm8Ygzguax2N3Fa/D/ag1WqbOprdI+uY6wMUl8/a2G
+iag==",
    "dev": true,
    "license": "Apache-2.0",
    "engines": {
```

```
      "node": "^12.22.0 || ^14.17.0 || >=16.0.0"
    },
    "funding": {
      "url": "https://opencollective.com/eslint"
    }
  },
  "node_modules/@eslint-community/regexpp": {
    "version": "4.12.1",
    "resolved": "https://registry.npmjs.org/@eslint-community/regexpp/-/regexpp-4.12.1.tgz",
    "integrity": "sha512-
CCZCDJuduB9OUkFkY2IgppNZMi2lBQgD2qzwXkEia16cge2pijY/aXi96CJMquDMn3nJdlPV1A5KrJ
EXwfLNzQ==",
    "dev": true,
    "license": "MIT",
    "engines": {
      "node": "^12.0.0 || ^14.0.0 || >=16.0.0"
    }
  },
  "node_modules/@eslint/config-array": {
    "version": "0.21.0",
    "resolved": "https://registry.npmjs.org/@eslint/config-array/-/config-array-0.21.0.tgz",
    "integrity": "sha512-
ENIdc4iLu0d93HeYirvKmrzshzofPw6VkZRKQGe9Nv46ZnWUzcF1xV01dcvEg/1wXUR61OmmlSfyey
O7EvjLxQ==",
    "dev": true,
    "license": "Apache-2.0",
    "dependencies": {
      "@eslint/object-schema": "^2.1.6",
      "debug": "^4.3.1",
      "minimatch": "^3.1.2"
    },
    "engines": {
      "node": "^18.18.0 || ^20.9.0 || >=21.1.0"
    }
  },
```

tailwind.config.ts

```
import type { Config } from "tailwindcss";

export default {
  darkMode: ["class"],
  content: ["./pages/**/*.{ts,tsx}", "./components/**/*.{ts,tsx}", "./app/**/*.{ts,tsx}",
"./src/**/*.{ts,tsx}"],
  prefix: "",
  theme: {
    container: {
      center: true,
      padding: "2rem",
      screens: {
        "2xl": "1400px",
      },
    },
    extend: {
      colors: {
        border: "hsl(var(--border))",
        input: "hsl(var(--input))",
        ring: "hsl(var(--ring))",
        background: "hsl(var(--background))",
        foreground: "hsl(var(--foreground))",
        primary: {
          DEFAULT: "hsl(var(--primary))",
          foreground: "hsl(var(--primary-foreground))",
        },
        secondary: {
          DEFAULT: "hsl(var(--secondary))",
          foreground: "hsl(var(--secondary-foreground))",
        },
        destructive: {
          DEFAULT: "hsl(var(--destructive))",
          foreground: "hsl(var(--destructive-foreground))",
        },
        muted: {
```

```
    DEFAULT: "hsl(var(--muted))",
    foreground: "hsl(var(--muted-foreground))",
   },
   accent: {
    DEFAULT: "hsl(var(--accent))",
    foreground: "hsl(var(--accent-foreground))",
   },
   popover: {
    DEFAULT: "hsl(var(--popover))",
    foreground: "hsl(var(--popover-foreground))",
   },
   card: {
    DEFAULT: "hsl(var(--card))",
    foreground: "hsl(var(--card-foreground))",
   },
   sidebar: {
    DEFAULT: "hsl(var(--sidebar-background))",
    foreground: "hsl(var(--sidebar-foreground))",
    primary: "hsl(var(--sidebar-primary))",
    "primary-foreground": "hsl(var(--sidebar-primary-foreground))",
    accent: "hsl(var(--sidebar-accent))",
    "accent-foreground": "hsl(var(--sidebar-accent-foreground))",
    border: "hsl(var(--sidebar-border))",
    ring: "hsl(var(--sidebar-ring))",
   },
  },
  borderRadius: {
   lg: "var(--radius)",
   md: "calc(var(--radius) - 2px)",
   sm: "calc(var(--radius) - 4px)",
  },
  keyframes: {
   "accordion-down": {
    from: {
     height: "0",
    },
    to: {
```

```
          height: "var(--radix-accordion-content-height)",
        },
      },
      "accordion-up": {
        from: {
          height: "var(--radix-accordion-content-height)",
        },
        to: {
          height: "0",
        },
      },
    },
    animation: {
      "accordion-down": "accordion-down 0.2s ease-out",
      "accordion-up": "accordion-up 0.2s ease-out",
    },
  },
 },
 plugins: [require("tailwindcss-animate")],
} satisfies Config;
```

package.json

```
{
 "name": "vite_react_shadcn_ts",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
  "dev": "vite",
  "build": "vite build",
  "build:dev": "vite build --mode development",
  "lint": "eslint .",
  "preview": "vite preview"
 },
```

```
"dependencies": {
  "@hookform/resolvers": "^3.10.0",
  "@radix-ui/react-accordion": "^1.2.11",
  "@radix-ui/react-alert-dialog": "^1.1.14",
  "@radix-ui/react-aspect-ratio": "^1.1.7",
  "@radix-ui/react-avatar": "^1.1.10",
  "@radix-ui/react-checkbox": "^1.3.2",
  "@radix-ui/react-collapsible": "^1.1.11",
  "@radix-ui/react-context-menu": "^2.2.15",
  "@radix-ui/react-dialog": "^1.1.14",
  "@radix-ui/react-dropdown-menu": "^2.1.15",
  "@radix-ui/react-hover-card": "^1.1.14",
  "@radix-ui/react-label": "^2.1.7",
  "@radix-ui/react-menubar": "^1.1.15",
  "@radix-ui/react-navigation-menu": "^1.2.13",
  "@radix-ui/react-popover": "^1.1.14",
  "@radix-ui/react-progress": "^1.1.7",
  "@radix-ui/react-radio-group": "^1.3.7",
  "@radix-ui/react-scroll-area": "^1.2.9",
  "@radix-ui/react-select": "^2.2.5",
  "@radix-ui/react-separator": "^1.1.7",
  "@radix-ui/react-slider": "^1.3.5",
  "@radix-ui/react-slot": "^1.2.3",
  "@radix-ui/react-switch": "^1.2.5",
  "@radix-ui/react-tabs": "^1.1.12",
  "@radix-ui/react-toast": "^1.2.14",
  "@radix-ui/react-toggle": "^1.1.9",
  "@radix-ui/react-toggle-group": "^1.1.10",
  "@radix-ui/react-tooltip": "^1.2.7",
  "@supabase/supabase-js": "^2.89.0",
  "@tanstack/react-query": "^5.83.0",
  "class-variance-authority": "^0.7.1",
  "clsx": "^2.1.1",
  "cmdk": "^1.1.1",
  "date-fns": "^3.6.0",
  "embla-carousel-react": "^8.6.0",
  "input-otp": "^1.4.2",
```

```
    "lucide-react": "^0.462.0",
    "next-themes": "^0.3.0",
    "qrcode.react": "^4.2.0",
    "react": "^18.3.1",
    "react-day-picker": "^8.10.1",
    "react-dom": "^18.3.1",
    "react-hook-form": "^7.61.1",
    "react-resizable-panels": "^2.1.9",
    "react-router-dom": "^6.30.1",
    "recharts": "^2.15.4",
    "sonner": "^1.7.4",
    "tailwind-merge": "^2.6.0",
    "tailwindcss-animate": "^1.0.7",
    "vaul": "^0.9.9",
    "zod": "^3.25.76"
  },
  "devDependencies": {
    "@eslint/js": "^9.32.0",
    "@tailwindcss/typography": "^0.5.16",
    "@types/node": "^22.16.5",
    "@types/react": "^18.3.23",
    "@types/react-dom": "^18.3.7",
    "@vitejs/plugin-react-swc": "^3.11.0",
    "autoprefixer": "^10.4.21",
    "eslint": "^9.32.0",
    "eslint-plugin-react-hooks": "^5.2.0",
    "eslint-plugin-react-refresh": "^0.4.20",
    "globals": "^15.15.0",
    "lovable-tagger": "^1.1.13",
    "postcss": "^8.5.6",
    "tailwindcss": "^3.4.17",
    "typescript": "^5.8.3",
    "typescript-eslint": "^8.38.0",
    "vite": "^8.0.1"
  }
}
```

StudentSignupForm. tsx

```
import { useState } from "react";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { studentSignupSchema, StudentSignupData } from "@/lib/validations";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { useAuth } from "@/hooks/useAuth";
import { useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { toast } from "sonner";
import { Loader2, User, Mail, Phone, Lock, IdCard } from "lucide-react";

export function StudentSignupForm() {
  const [isLoading, setIsLoading] = useState(false);
  const { signUp } = useAuth();
  const navigate = useNavigate();

  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm<StudentSignupData>({
    resolver: zodResolver(studentSignupSchema),
  });

  const onSubmit = async (data: StudentSignupData) => {
    setIsLoading(true);
    try {
      const { error, userId } = await signUp(data.email, data.password, {
        role: "student",
        full_name: data.fullName,
        phone: data.phone,
        appar_id: data.apparId || null,
```

```
        status: "pending",
      });

      if (error) {
        if (error.message.includes("already registered")) {
          toast.error("This email is already registered. Please login instead.");
        } else {
          toast.error(error.message);
        }
        return;
      }

      if (userId) {
        const { error: otpError } = await supabase.functions.invoke("send-otp", {
          body: { userId, email: data.email },
        });

        if (otpError) {
          console.error("Failed to send OTP:", otpError);
          toast.error("Account created but failed to send verification code. Please login and request a new
code.");
          navigate("/login");
          return;
        }

        toast.success("Account created! Please verify your email.");
        navigate("/verify-otp", { state: { userId, email: data.email } });
      }
    } catch (error) {
      toast.error("An unexpected error occurred. Please try again.");
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="space-y-5">
```

```
<div className="space-y-2">
  <Label htmlFor="fullName" className="flex items-center gap-2">
    <User className="h-4 w-4 text-muted-foreground" />
    Full Name
  </Label>
  <Input
    id="fullName"
    placeholder="John Doe"
    {...register("fullName")}
    className={errors.fullName ? "border-destructive" : ""}
  />
  {errors.fullName && (
    <p className="text-sm text-destructive">{errors.fullName.message}</p>
  )}
</div>

<div className="space-y-2">
  <Label htmlFor="email" className="flex items-center gap-2">
    <Mail className="h-4 w-4 text-muted-foreground" />
    Email
  </Label>
  <Input
    id="email"
    type="email"
    placeholder="john@example.com"
    {...register("email")}
    className={errors.email ? "border-destructive" : ""}
  />
  {errors.email && (
    <p className="text-sm text-destructive">{errors.email.message}</p>
  )}
</div>

<div className="space-y-2">
  <Label htmlFor="phone" className="flex items-center gap-2">
    <Phone className="h-4 w-4 text-muted-foreground" />
    Phone Number
```

```
  </Label>
  <Input
    id="phone"
    type="tel"
    placeholder="+1 234 567 8900"
    {...register("phone")}
    className={errors.phone ? "border-destructive" : ""}
  />
  {errors.phone && (
    <p className="text-sm text-destructive">{errors.phone.message}</p>
  )}
</div>

<div className="space-y-2">
  <Label htmlFor="password" className="flex items-center gap-2">
    <Lock className="h-4 w-4 text-muted-foreground" />
    Password
  </Label>
  <Input
    id="password"
    type="password"
    placeholder="••••••••"
    {...register("password")}
    className={errors.password ? "border-destructive" : ""}
  />
  {errors.password && (
    <p className="text-sm text-destructive">{errors.password.message}</p>
  )}
  <p className="text-xs text-muted-foreground">
    Min 8 characters, 1 number, 1 symbol
  </p>
</div>

<div className="space-y-2">
  <Label htmlFor="apparId" className="flex items-center gap-2">
    <IdCard className="h-4 w-4 text-muted-foreground" />
    APPAR ID <span className="text-muted-foreground">(Optional)</span>
```

```
      </Label>
      <Input
        id="apparId"
        placeholder="Enter your APPAR ID"
        {...register("apparId")}
      />
    </div>

    <Button type="submit" variant="hero" size="lg" className="w-full" disabled={isLoading}>
      {isLoading ? (
        <>
          <Loader2 className="mr-2 h-4 w-4 animate-spin" />
          Creating Account...
        </>
      ) : (
        "Create Student Account"
      )}
    </Button>
  </form>
 );
}
```

CompanySignupForm

```
import { useState } from "react";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { companySignupSchema, CompanySignupData } from "@/lib/validations";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { useAuth } from "@/hooks/useAuth";
import { useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { toast } from "sonner";
import { Loader2, Briefcase, Mail, Phone, Lock, Globe } from "lucide-react";

export function CompanySignupForm() {
  const [isLoading, setIsLoading] = useState(false);
  const { signUp } = useAuth();
  const navigate = useNavigate();

  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm<CompanySignupData>({
    resolver: zodResolver(companySignupSchema),
  });

  const onSubmit = async (data: CompanySignupData) => {
    setIsLoading(true);
    try {
      const { error, userId } = await signUp(data.officialEmail, data.password, {
        role: "company",
        company_name: data.companyName,
        phone: data.phone,
        website: data.website || null,
        status: "pending",
      });

      if (error) {
        if (error.message.includes("already registered")) {
```

```
        toast.error("This email is already registered. Please login instead.");
      } else {
        toast.error(error.message);
      }
      return;
    }

    if (userId) {
      const { error: otpError } = await supabase.functions.invoke("send-otp", {
        body: { userId, email: data.officialEmail },
      });

      if (otpError) {
        toast.error("Account created but failed to send verification code.");
        navigate("/login");
        return;
      }

      toast.success("Account created! Please verify your email.");
      navigate("/verify-otp", { state: { userId, email: data.officialEmail } });
    }
  } catch (error) {
    toast.error("An unexpected error occurred. Please try again.");
  } finally {
    setIsLoading(false);
  }
};

return (
  <form onSubmit={handleSubmit(onSubmit)} className="space-y-5">
    <div className="space-y-2">
      <Label htmlFor="companyName" className="flex items-center gap-2">
        <Briefcase className="h-4 w-4 text-muted-foreground" />
        Company Name
      </Label>
      <Input
        id="companyName"
        placeholder="Acme Corporation"
        {...register("companyName")}
        className={errors.companyName ? "border-destructive" : ""}
```

```
  />
  {errors.companyName && (
    <p className="text-sm text-destructive">{errors.companyName.message}</p>
  )}
</div>

<div className="space-y-2">
  <Label htmlFor="officialEmail" className="flex items-center gap-2">
    <Mail className="h-4 w-4 text-muted-foreground" />
    Official Email
  </Label>
  <Input
    id="officialEmail"
    type="email"
    placeholder="hr@company.com"
    {...register("officialEmail")}
    className={errors.officialEmail ? "border-destructive" : ""}
  />
  {errors.officialEmail && (
    <p className="text-sm text-destructive">{errors.officialEmail.message}</p>
  )}
</div>

<div className="space-y-2">
  <Label htmlFor="phone" className="flex items-center gap-2">
    <Phone className="h-4 w-4 text-muted-foreground" />
    Phone Number
  </Label>
  <Input
    id="phone"
    type="tel"
    placeholder="+1 234 567 8900"
    {...register("phone")}
    className={errors.phone ? "border-destructive" : ""}
  />
  {errors.phone && (
    <p className="text-sm text-destructive">{errors.phone.message}</p>
  )}
</div>
```

```
<div className="space-y-2">
  <Label htmlFor="password" className="flex items-center gap-2">
    <Lock className="h-4 w-4 text-muted-foreground" />
    Password
  </Label>
  <Input
    id="password"
    type="password"
    placeholder="••••••••"
    {...register("password")}
    className={errors.password ? "border-destructive" : ""}
  />
  {errors.password && (
    <p className="text-sm text-destructive">{errors.password.message}</p>
  )}
  <p className="text-xs text-muted-foreground">
    Min 8 characters, 1 number, 1 symbol
  </p>
</div>

<div className="space-y-2">
  <Label htmlFor="website" className="flex items-center gap-2">
    <Globe className="h-4 w-4 text-muted-foreground" />
    Website <span className="text-muted-foreground">(Optional)</span>
  </Label>
  <Input
    id="website"
    type="url"
    placeholder="https://company.com"
    {...register("website")}
  />
  {errors.website && (
    <p className="text-sm text-destructive">{errors.website.message}</p>
  )}
</div>

<Button type="submit" variant="hero" size="lg" className="w-full" disabled={isLoading}>
  {isLoading ? (
    <>
      <Loader2 className="mr-2 h-4 w-4 animate-spin" />
```

```
          Creating Account...
        </>
      ) : (
        "Create Company Account"
      )}
    </Button>
  </form>
 );
}
```

InstituteSignupForm

```
import { useState } from "react";
import { useForm } from "react-hook-form";
import { zodResolver } from "@hookform/resolvers/zod";
import { instituteSignupSchema, InstituteSignupData } from "@/lib/validations";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { useAuth } from "@/hooks/useAuth";
import { useNavigate } from "react-router-dom";
import { supabase } from "@/integrations/supabase/client";
import { toast } from "sonner";
import { Loader2, Building2, Mail, Phone, Lock, MapPin } from "lucide-react";

export function InstituteSignupForm() {
  const [isLoading, setIsLoading] = useState(false);
  const { signUp } = useAuth();
  const navigate = useNavigate();

  const {
    register,
```

```
  handleSubmit,
  formState: { errors },
} = useForm<InstituteSignupData>({
  resolver: zodResolver(instituteSignupSchema),
});

const onSubmit = async (data: InstituteSignupData) => {
  setIsLoading(true);
  try {
    const { error, userId } = await signUp(data.email, data.password, {
      role: "institute",
      institute_name: data.instituteName,
      phone: data.phone,
      address: data.address || null,
      status: "pending",
    });

    if (error) {
      if (error.message.includes("already registered")) {
        toast.error("This email is already registered. Please login instead.");
      } else {
        toast.error(error.message);
      }
      return;
    }

    if (userId) {
      const { error: otpError } = await supabase.functions.invoke("send-otp", {
        body: { userId, email: data.email },
      });

      if (otpError) {
        toast.error("Account created but failed to send verification code.");
        navigate("/login");
```

```
        return;
      }

      toast.success("Account created! Please verify your email.");
      navigate("/verify-otp", { state: { userId, email: data.email } });
    }
  } catch (error) {
    toast.error("An unexpected error occurred. Please try again.");
  } finally {
    setIsLoading(false);
  }
};

return (
  <form onSubmit={handleSubmit(onSubmit)} className="space-y-5">
    <div className="space-y-2">
      <Label htmlFor="instituteName" className="flex items-center gap-2">
        <Building2 className="h-4 w-4 text-muted-foreground" />
        Institute Name
      </Label>
      <Input
        id="instituteName"
        placeholder="Harvard University"
        {...register("instituteName")}
        className={errors.instituteName ? "border-destructive" : ""}
      />
      {errors.instituteName && (
        <p className="text-sm text-destructive">{errors.instituteName.message}</p>
      )}
    </div>

    <div className="space-y-2">
      <Label htmlFor="email" className="flex items-center gap-2">
        <Mail className="h-4 w-4 text-muted-foreground" />
```

```
    Official Email
  </Label>
  <Input
    id="email"
    type="email"
    placeholder="admin@university.edu"
    {...register("email")}
    className={errors.email ? "border-destructive" : ""}
  />
  {errors.email && (
    <p className="text-sm text-destructive">{errors.email.message}</p>
  )}
</div>

<div className="space-y-2">
  <Label htmlFor="phone" className="flex items-center gap-2">
    <Phone className="h-4 w-4 text-muted-foreground" />
    Phone Number
  </Label>
  <Input
    id="phone"
    type="tel"
    placeholder="+1 234 567 8900"
    {...register("phone")}
    className={errors.phone ? "border-destructive" : ""}
  />
  {errors.phone && (
    <p className="text-sm text-destructive">{errors.phone.message}</p>
  )}
</div>

<div className="space-y-2">
  <Label htmlFor="password" className="flex items-center gap-2">
    <Lock className="h-4 w-4 text-muted-foreground" />
```

```
          Password
        </Label>
        <Input
          id="password"
          type="password"
          placeholder="••••••••"
          {...register("password")}
          className={errors.password ? "border-destructive" : ""}
        />
        {errors.password && (
          <p className="text-sm text-destructive">{errors.password.message}</p>
        )}
        <p className="text-xs text-muted-foreground">
          Min 8 characters, 1 number, 1 symbol
        </p>
      </div>

      <div className="space-y-2">
        <Label htmlFor="address" className="flex items-center gap-2">
          <MapPin className="h-4 w-4 text-muted-foreground" />
          Address <span className="text-muted-foreground">(Optional)</span>
        </Label>
        <Input
          id="address"
          placeholder="123 University Ave, City"
          {...register("address")}
        />
      </div>

      <Button type="submit" variant="hero" size="lg" className="w-full"
disabled={isLoading}>
        {isLoading ? (
          <>
            <Loader2 className="mr-2 h-4 w-4 animate-spin" />
```

```
          Creating Account...
        </>
      ) : (
        "Create Institute Account"
      )}
    </Button>
  </form>
 );
}
```

CHAPTER 6

6. SYSTEM TEST

Testing a blockchain credentialing system requires a vastly different approach than testing a traditional centralized application. Because smart contracts are immutable once deployed and data is permanently etched onto a public ledger, the cost of failure is absolute. Your testing strategy must be aggressive, anticipating malicious actors, network congestion, and edge-case user errors. Do not treat these phases as a checklist; treat them as an adversarial attack on your own architecture.

6.1 UNIT TESTING

Unit testing in a blockchain environment is not a box you check for a grade. If your smart contract logic is flawed at the granular level, your entire credentialing system is a liability. For this project, unit testing must aggressively isolate every function within your smart contracts and backend scripts to prove they execute flawlessly under both ideal and adversarial conditions. You cannot patch a deployed smart contract easily, so your unit tests are your absolute first line of defense.

Focus your testing heavily on the core cryptographic mechanics. You must write tests specifically validating your generateHash function. Does it output a consistent, deterministic SHA-256 hash every single time for the exact same document payload? Test your state-changing functions like issueCredential and revokeCredential. You need to assert that only an authorized institutional wallet address can trigger these functions. If your tests do not explicitly check for unauthorized access attempts and ensure the transaction reverts, you have a massive blind spot in your security model.

Furthermore, write tests for your off-chain logic. Your backend script that handles the initial file upload must be tested to ensure it never attempts to store the actual document payload on the blockchain. Test the script's ability to reject malformed or oversized files before they ever reach the hashing phase. You also need to test your gas optimization logic. If you are using bytes32 instead of string to store hashes, write a unit test to measure and assert the exact gas cost of that specific function execution. Do not assume your code is efficient; prove it mathematically in the test environment. Run these tests in a continuous integration pipeline. A minor bug in a traditional database is a quick hotfix. A minor bug in a blockchain registry permanently compromises the academic record of every student in your system.

6.2 INTERGRATION TESTING

Integration testing is where the illusions of your system architecture shatter. It is easy to make a smart contract work in isolation and an IPFS node work locally. The reality is that your blockchain credential system lives or dies based on the fragile handshake between your off-chain storage, your backend application, and the on-chain ledger. You need to rigorously test these intersections because this is exactly where data gets lost, misrouted, or corrupted during execution.

Start by testing the pipeline between your file upload service and IPFS. When an institution uploads a credential, does the system reliably generate the CID (Content Identifier) from the IPFS pinning service? More importantly, test the failure states. What happens if the IPFS pinning service experiences downtime or times out? Your integration tests must prove that the backend gracefully handles the error and prevents a partial issuance where a hash is recorded on-chain without the corresponding document successfully pinning off-chain.

Next, target the integration between your backend logic—utilizing Web3.js or Ethers.js—and the RPC node connecting you to the blockchain network. You must test the transaction signing and submission process. When the backend submits the hash to the smart contract, does the network successfully receive it? You need to verify that your application actively listens for the CredentialIssued event emitted by the smart contract. If your backend cannot reliably catch that event and update the database or user interface, your integration is broken. Do not assume node providers will always respond instantly. Write integration tests that simulate network congestion, dropped connections, and delayed block confirmations to ensure the system remains stable.

6.3 FUNCTIONAL TESTING

Functional testing strips away the technical vanity of your blockchain implementation and asks one brutal question: does this system actually solve the real-world problem it was built for? You are building a credentialing platform, not a decentralized tech demo. Therefore, your functional testing must map directly to the strict business requirements of universities, students, and corporate recruiters. If the system technically works but is completely unusable by a non-technical HR representative, your project is an operational failure.

You must test the complete user journeys against your defined use cases. For the institution,

test the batch issuance workflow. Can an administrator upload a CSV file of 500 student records, and does the system correctly process, hash, and issue them in a single, cost-effective transaction without timing out? For the student, test the wallet integration and access. Can they easily retrieve their digital credential and generate a shareable QR code without needing to understand public-key cryptography or manage gas fees?

The most critical functional tests belong to the verification process. Test the drag-and-drop verification portal interface. When a verifier uploads a legitimate, unaltered PDF, does the system clearly and instantly return a positive authentication status? Conversely, take a legitimate PDF, alter a single pixel or change a single grade off-chain, and run it through the verifier again. Your functional tests must unequivocally prove that the system flags it as tampered every single time. Stop relying on technical jargon to excuse poor user experience. The functional testing phase is where you ensure the system provides a seamless, binary result to the end user. If a recruiter has to read a manual to verify a degree, your design is flawed.

6.4 SYSTEM TESTING

System testing evaluates your credentialing platform as a fully integrated, cohesive entity deployed in an environment that heavily mirrors production. This is no longer about testing isolated functions or API endpoints; this is about exposing your entire architecture to realistic loads, unstable network conditions, and end-to-end operational workflows. You must deploy your complete stack—smart contracts on a live testnet like Sepolia, active IPFS storage nodes, and the hosted front-end application—and rigorously test the macro behavior of the system.

Your primary focus here must be performance and scalability under extreme stress. You claim your architecture optimizes gas costs and handles high volumes without storing heavy data on-chain. System testing is where you prove that claim. Run automated scripts that simulate an entire university graduating class. Attempt to process and issue 10,000 credentials simultaneously. Monitor the transaction latency, the absolute gas fees incurred in native testnet tokens, and the server response times. If your system bottlenecks, crashes, or incurs exponentially higher fees than projected, your architecture requires immediate refactoring.

Additionally, system testing must validate data consistency across all layers over extended timeframes. Issue a batch of credentials, wait several days, and then simulate a recruiter verifying them. Does the IPFS caching layer hold up? Are the documents still accessible? You also need to test system recovery. Force a crash on your backend server mid-transaction while a batch issuance is pending. Does the system recover cleanly upon reboot, or do you end up

with orphaned hashes on the blockchain and missing files on IPFS? Prove your application handles catastrophic interruptions cleanly.

6.5 WHITE-BOX TESTING

White-box testing requires you to strip away the user interface and ruthlessly interrogate the internal structure, logic, and code of your application. Because your project deals with permanent cryptographic records and decentralized ledgers, your code must be airtight. You need to examine the specific Solidity code in your smart contracts and the backend routing logic with full transparency and zero assumptions.

Start with a line-by-line security audit of your smart contracts. You are checking for known vulnerabilities that routinely destroy blockchain projects. Test your access control modifiers strictly. You must write test scripts that explicitly attempt to bypass these controls from unauthorized wallets to prove your logic holds firm against unauthorized issuance or revocation. Analyze your data structures. Are you handling the 32-byte hash arrays efficiently? Map out every single execution path for the verification algorithm to determine the exact computational complexity. If your code includes redundant loops or unnecessary state reads, you are wasting gas and building an inefficient system.

You must also evaluate code coverage using automated static analysis tools. Run metrics to verify exactly what percentage of your codebase is executed during your test suites. If your white-box testing reveals that significant portions of your backend logic or smart contract conditional statements are never hit during normal operation, you have dead code or untested edge cases that represent a massive operational risk. Expose your own blind spots here, refactor the inefficient paths, and remove unnecessary complexities before a malicious actor exploits them on the mainnet.

6.6 BLACK-BOX TESTING

Black-box testing forces you to look at your credentialing system exactly how an end-user or a hostile attacker sees it—without any knowledge of the underlying code or architecture. You are strictly testing the inputs and the resulting outputs. In a system specifically designed to guarantee authenticity and prevent forgery, black-box testing must be highly adversarial. You need to actively try to deceive, overload, and break your own platform.

Focus heavily on input validation and edge cases from the user interface. What happens if a

compromised institutional account tries to upload a massive 5GB video file instead of a standard PDF certificate? What happens if they upload a file containing malicious executable code disguised as a document? Your black-box tests must ensure the front-end and backend cleanly reject these anomalous inputs without crashing the server or exposing internal stack traces. Test your rate limits aggressively to ensure your backend cannot be taken down by a simple denial-of-service attack flooding the verification endpoint.

Next, attempt to break the verification logic entirely from the outside. Create a valid credential using the standard workflow. Then, attempt to manually submit a transaction directly to the blockchain testnet, bypassing your UI completely, to alter that specific hash or revoke it using an unauthorized wallet. Your system must block this at the protocol level. Take a valid credential, change the file extension from PDF to a different format, and push it through the verifier. Does it fail securely? Black-box testing proves that regardless of how unpredictable or malicious the external input is, your system remains a mathematically enforced fortress.

CHAPTER 7

7. OUTPUT SCREEN

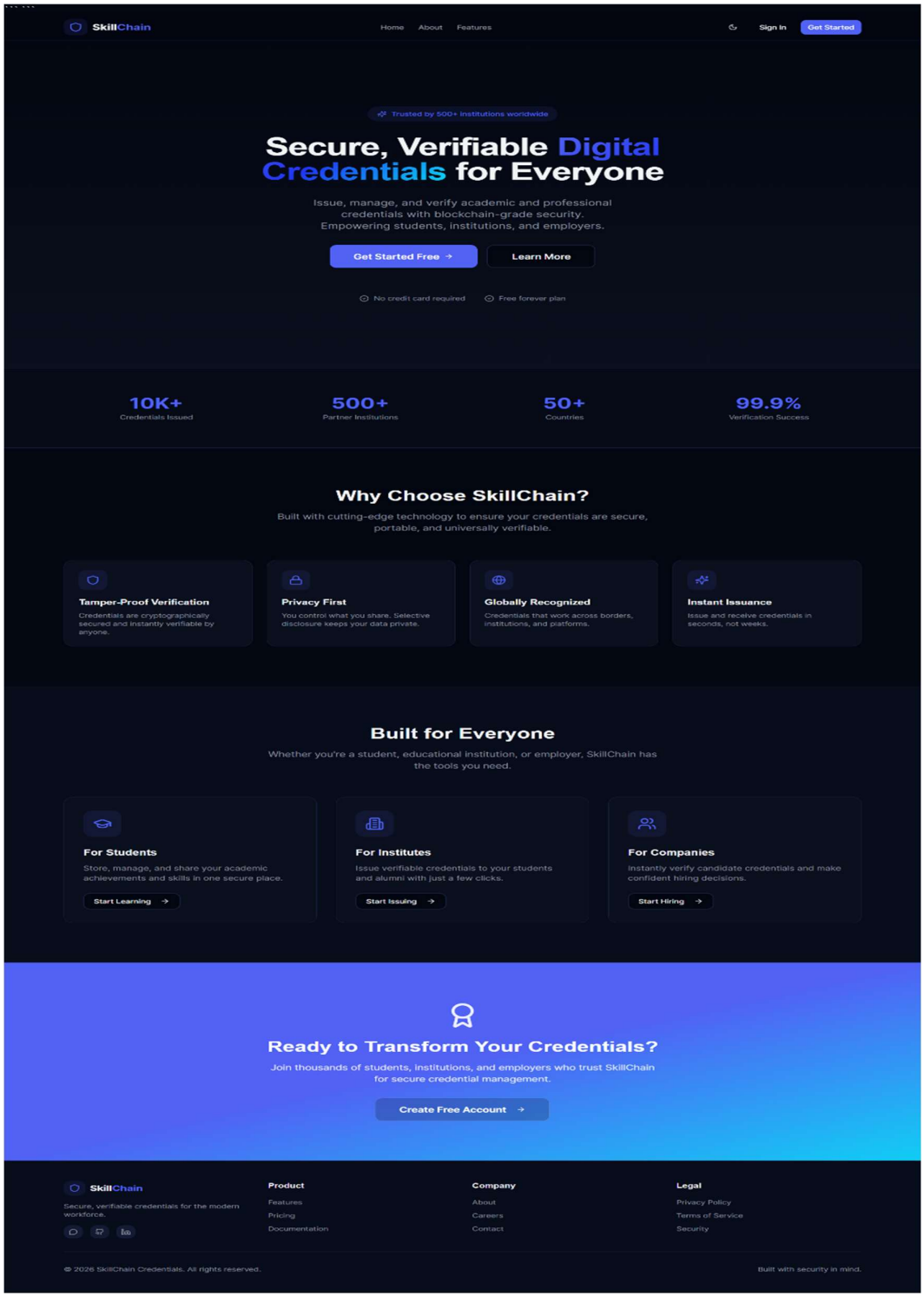


FIG:08- WEBSITE, FRONT-END

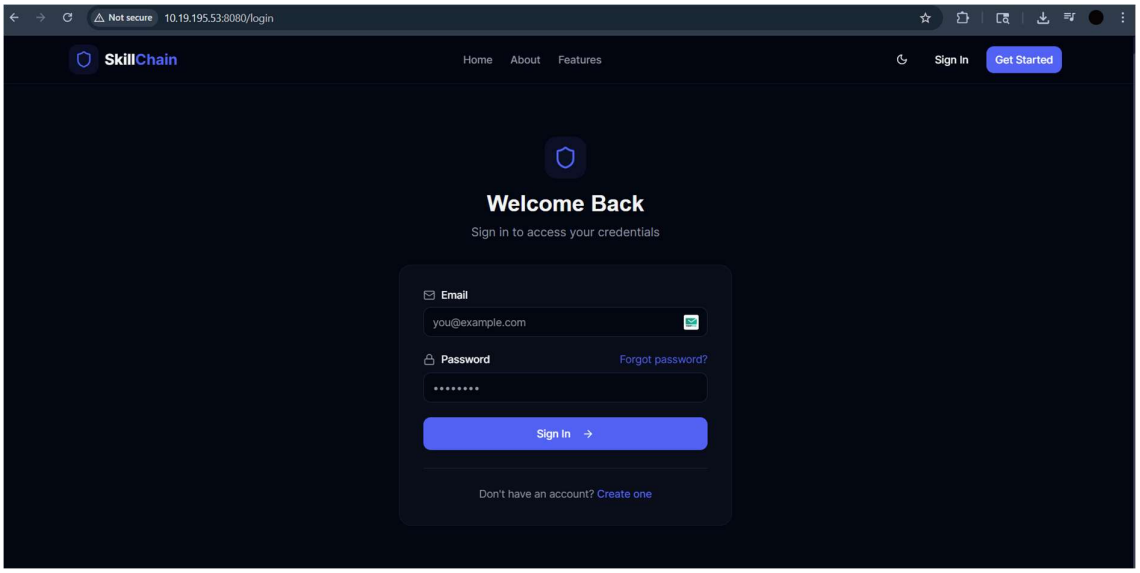


FIG:09 LOGIN-PAGE

SIGN-UP PAGE STUDENT

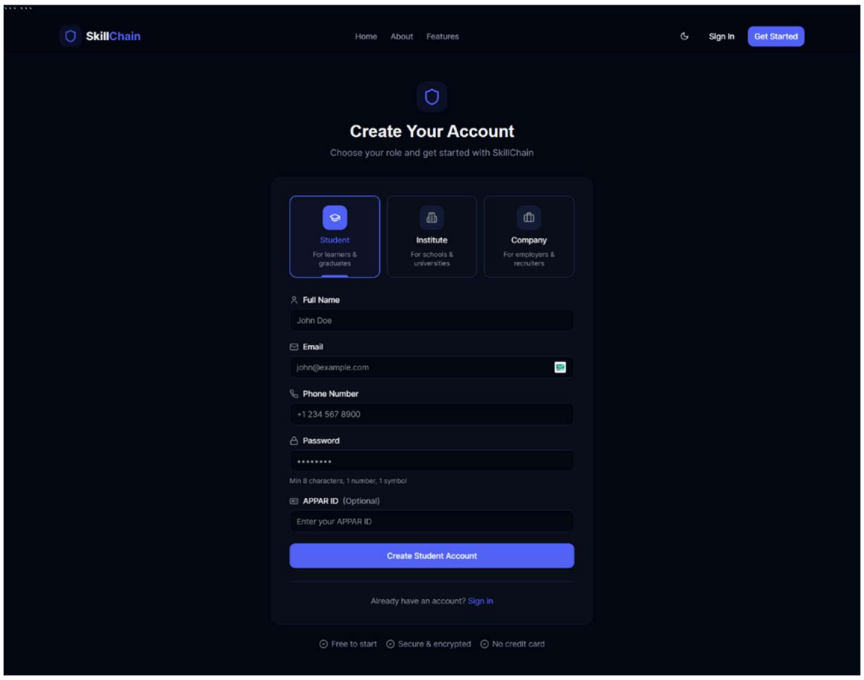


FIG:10 – STUDENT SIGN-UP PAGE

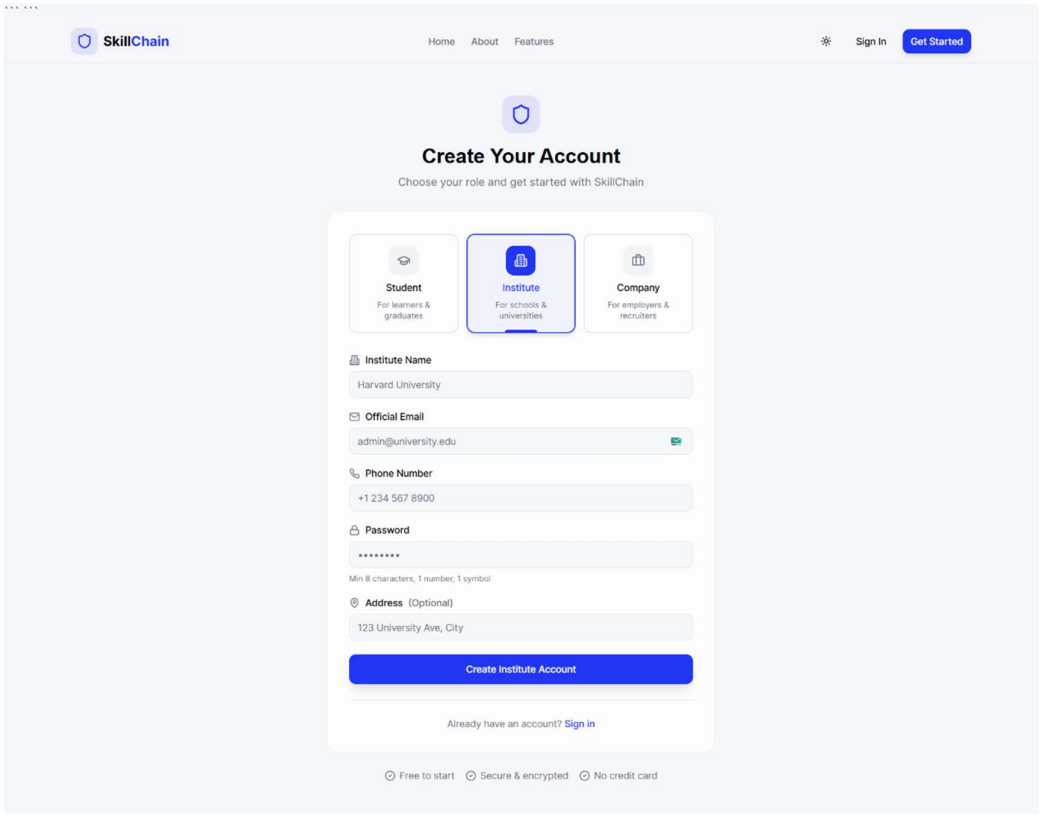


FIG:11 - SIGN-UP PAGE INSTITUTE

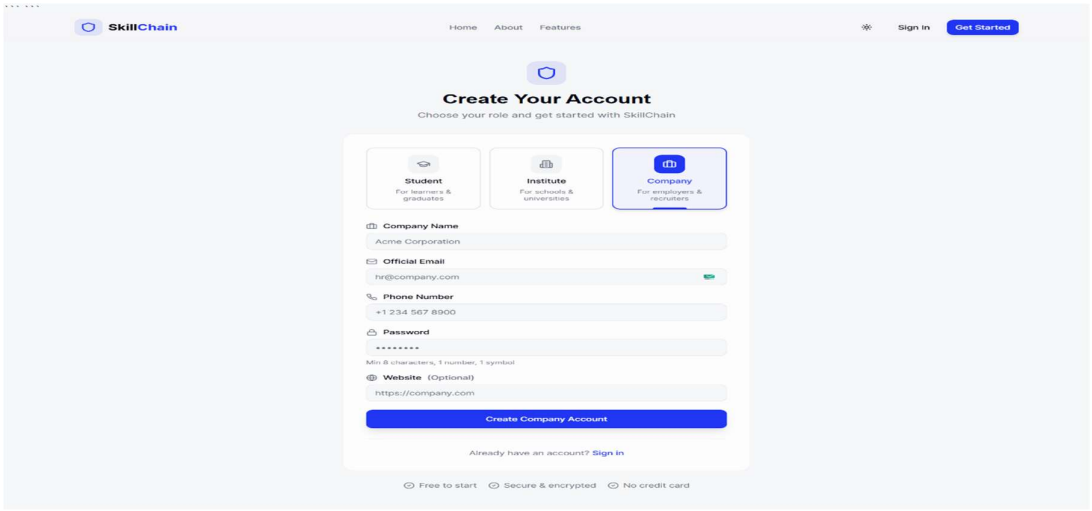


FIG:12 - SIGN-UP PAGE COMPANY

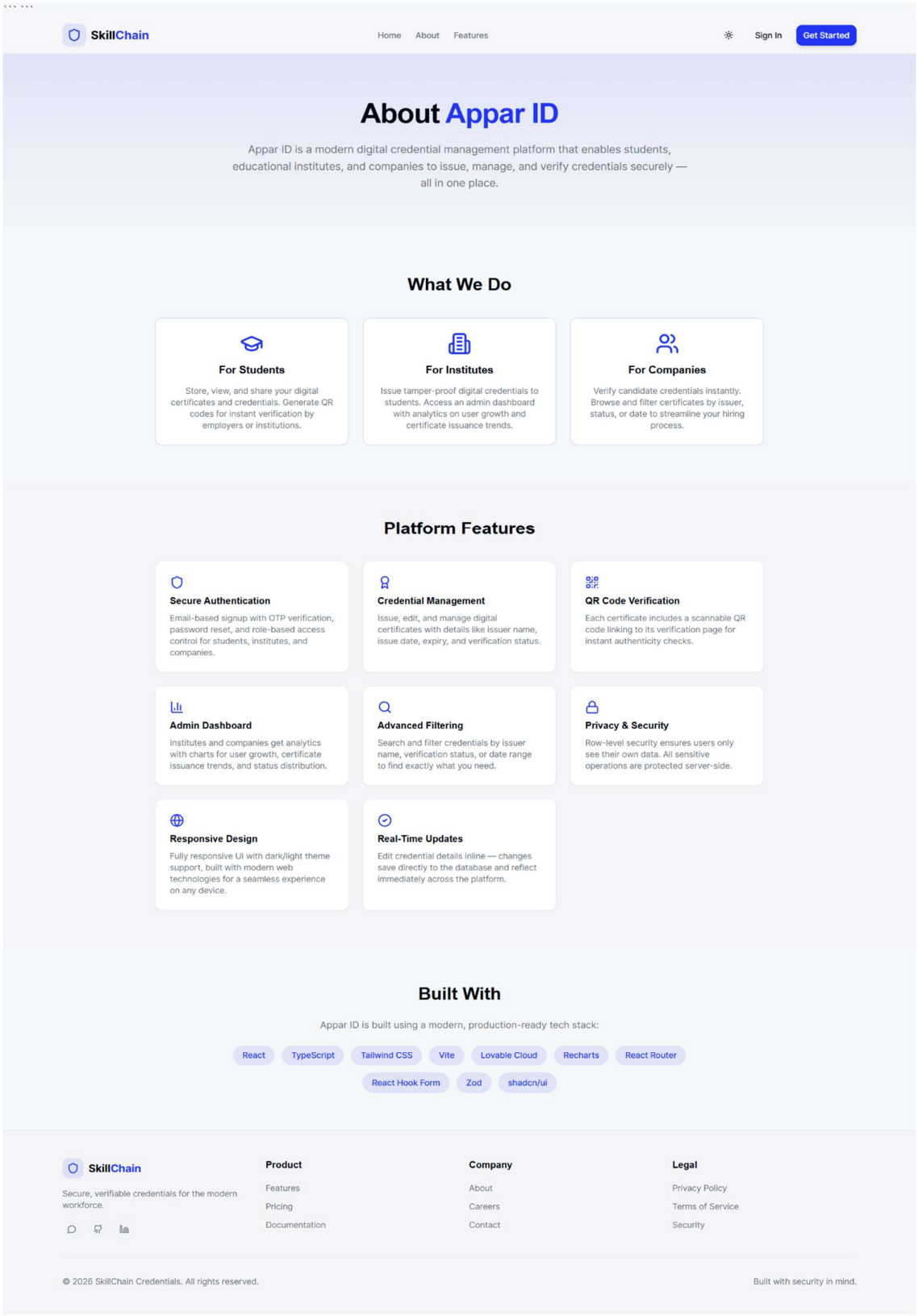


FIG:13 -ABOUT PAGE

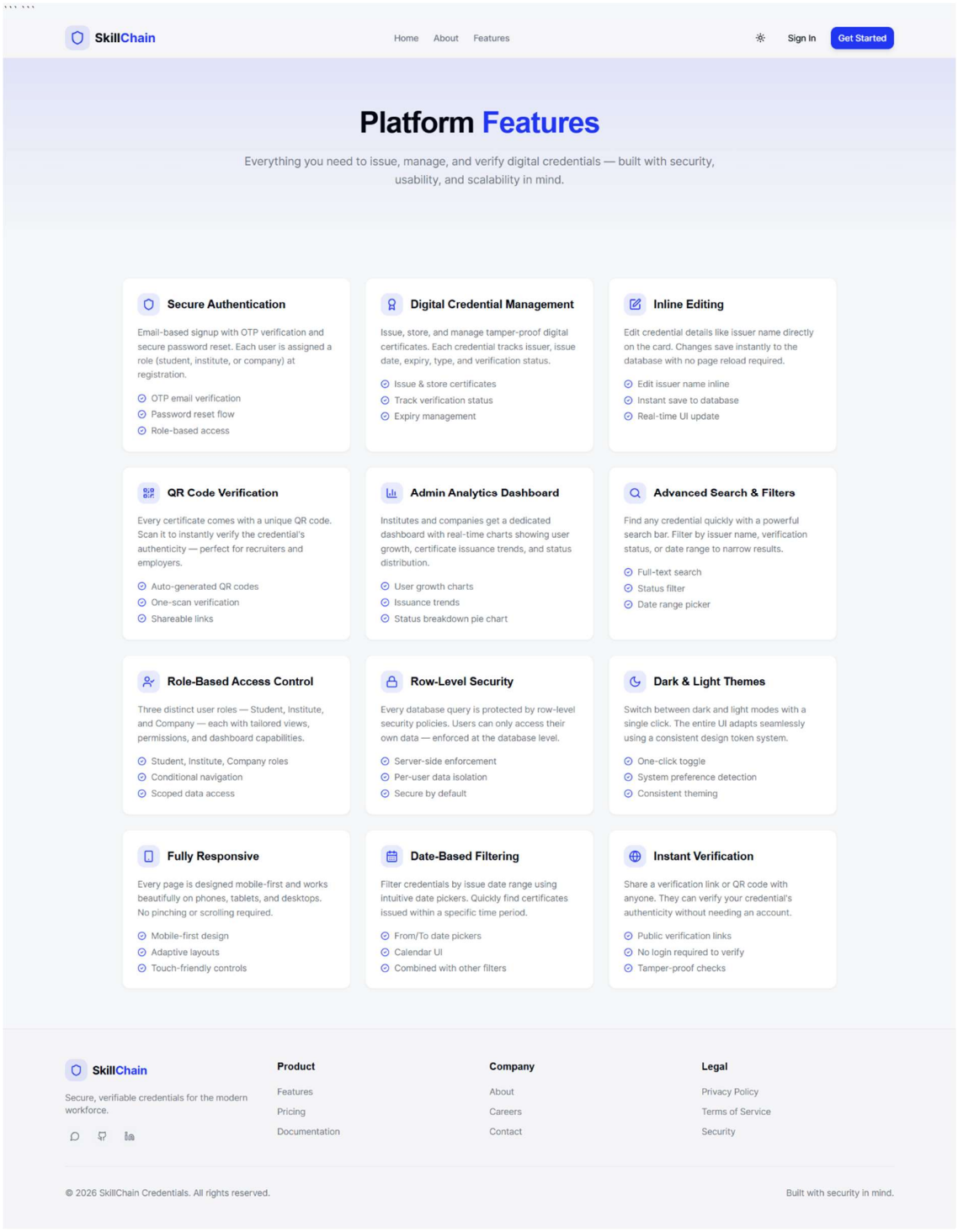


FIG:14 -FEATURE'S PAGE

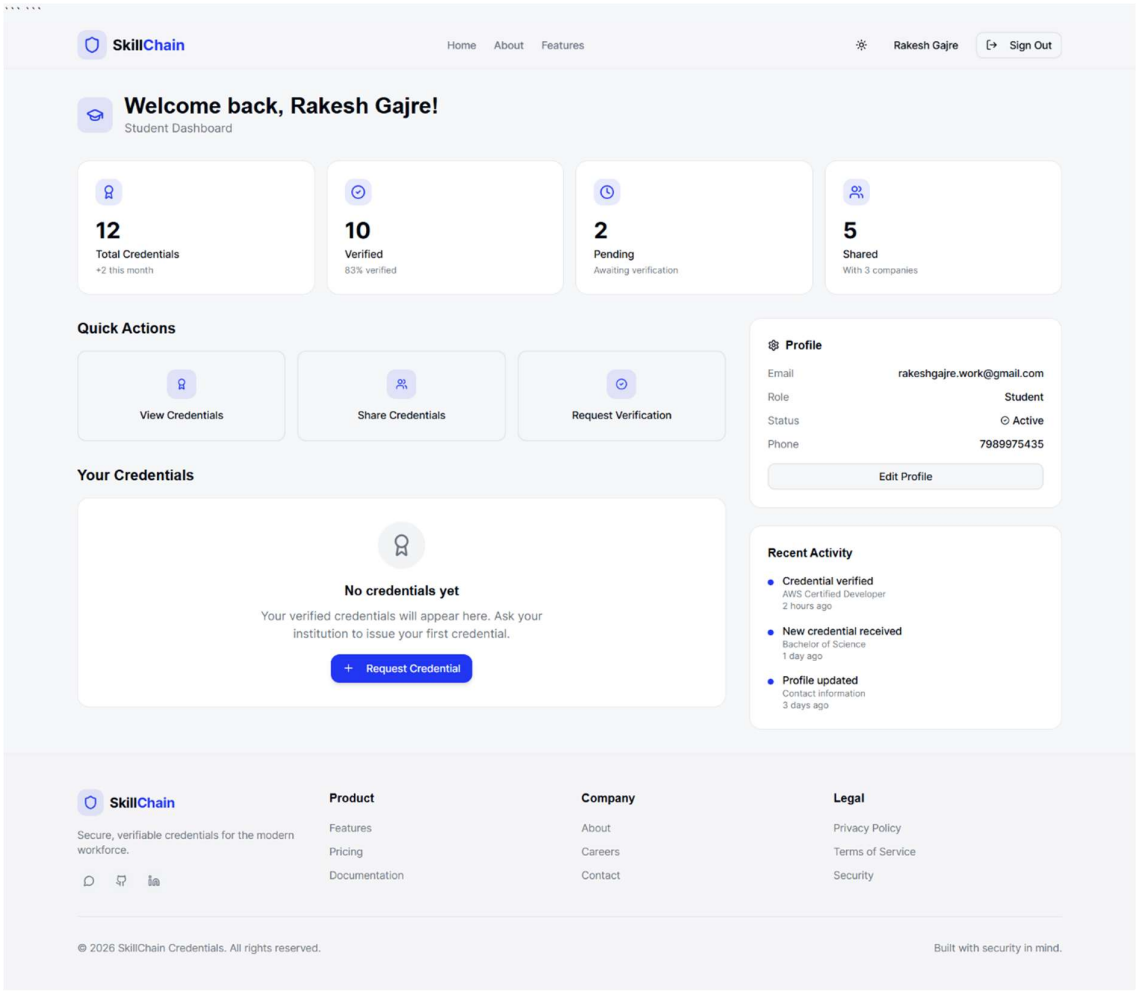
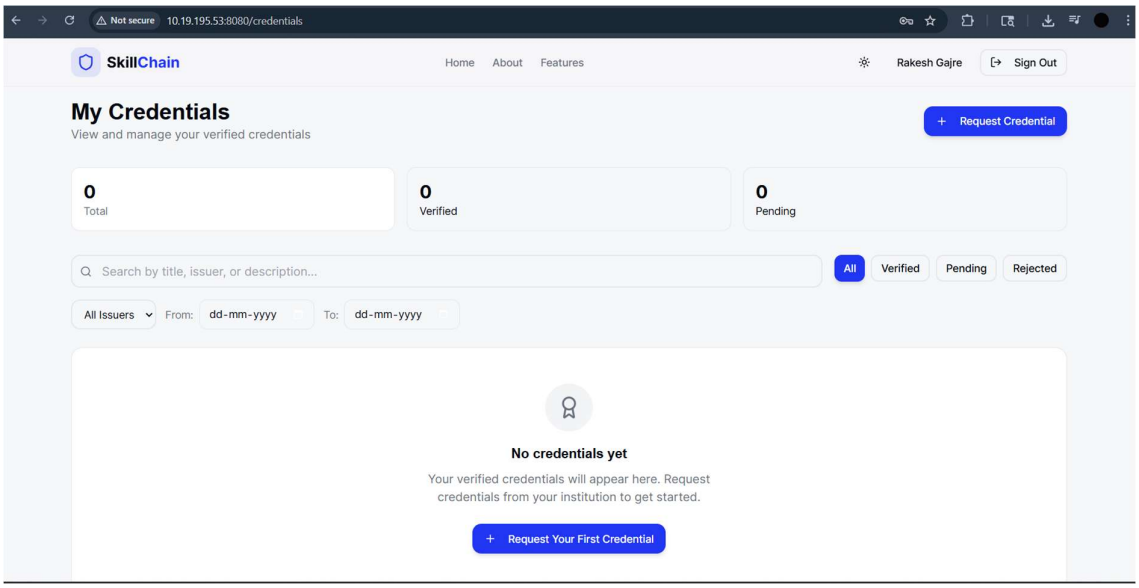


FIG:15 -AFTER LOGIN --PAGE



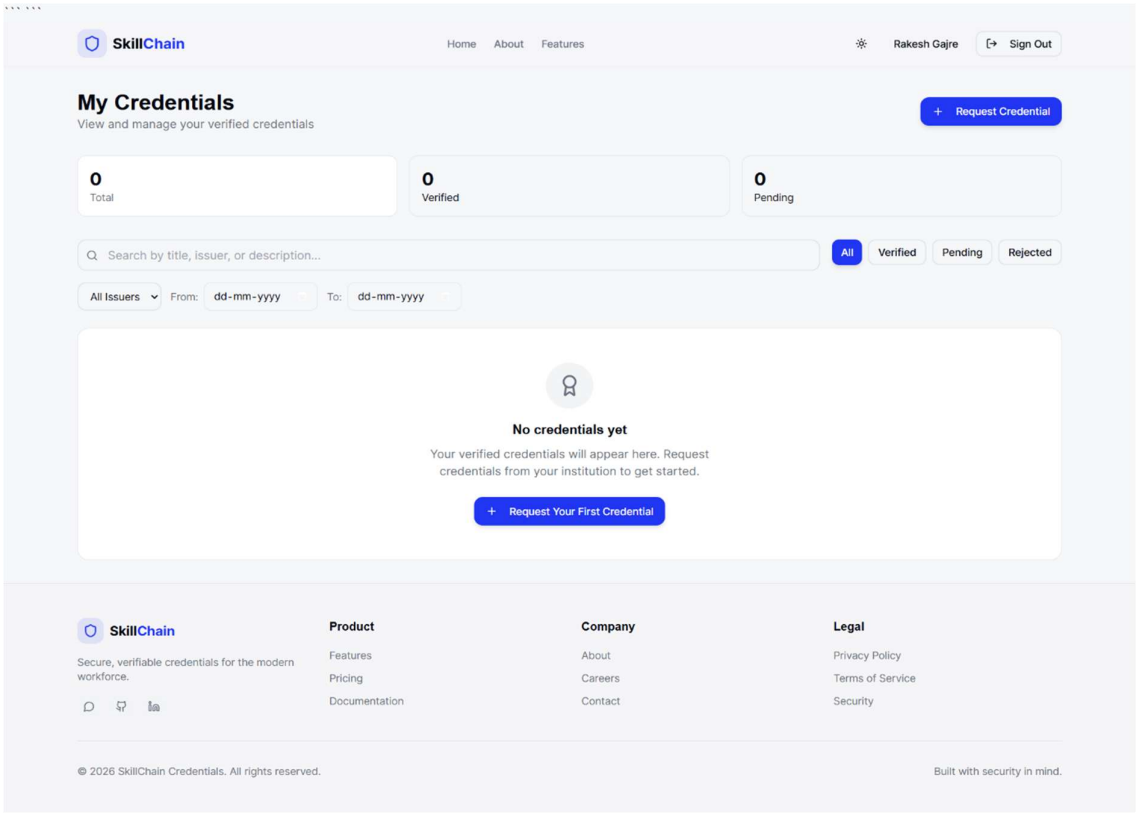


FIG:17 -MY CREDENTIALS

8. CONCLUSION

The traditional academic credentialing system is fundamentally broken. It relies on slow, centralized institutional databases that act as permanent bottlenecks for students and recruiters. While many propose blockchain as a solution, the naive approach of storing complete personal records on a public ledger is a severe architectural mistake. It violates data privacy and ignores the reality of scaling network costs.

This project succeeded because it abandoned the hype and implemented a pragmatic, hybrid architecture. By storing the actual credential documents off-chain using IPFS and anchoring only the mathematically irreversible SHA-256 hashes on the blockchain, the system achieves immutable security while strictly protecting user privacy. The defining technical achievement of this project is its cost optimization. By implementing Merkle tree batching, the system aggregates hundreds of credentials into a single on-chain transaction. This proves that decentralized verification is commercially viable for large institutions, dropping the issuance cost from prohibitive single transaction fees to mere pennies per record. The functional testing mathematically eliminated the possibility of backdating or off-chain tampering. If a document is altered by a single pixel, the hash mismatch immediately flags it as invalid.

The system is not without real world limitations. Mainstream adoption of digital wallets remains a user experience hurdle for the average student. Furthermore, off-chain storage still relies on the persistence of IPFS pinning services to prevent link rot. Despite these hurdles, the core thesis is proven. This architecture successfully shifts the burden of credential verification from slow institutional trust to instant cryptographic math. The system removes the issuing university from the verification loop entirely, giving permanent ownership of the academic record back to the student while providing recruiters with a frictionless, tamper proof validation tool.

9. FUTURE ENHANCEMENTS

While the current architecture successfully solves the fundamental problems of cost, security, and decentralized verification, it is a foundational layer. To scale this into a globally adopted, enterprise-grade standard, the system requires several advanced cryptographic and infrastructural upgrades. The following enhancements represent the immediate roadmap for the next iteration of the Skillchain Digital Credentialing System.

9.1 Zero-Knowledge Proofs (ZKPs) for Selective Disclosure Currently, a verifier must see the entire credential to verify its hash. We will implement Zero-Knowledge Proofs (specifically zk-SNARKs) to allow students to prove specific claims without revealing the underlying document. For example, a student could cryptographically prove to an employer that their GPA is above 3.5, or that they hold a valid medical license, without exposing their exact grades, graduation year, or personal address. This is the ultimate execution of data privacy.

9.2 Integration with W3C Decentralized Identifiers (DIDs) Relying on raw blockchain wallet addresses (e.g., 0x...) for institutional identity is poor UX and difficult to manage if private keys are compromised. The system will be upgraded to support standard Decentralized Identifiers (DIDs). This allows institutions to tie their issuing authority to a universally recognizable, rotatable digital identity, ensuring long-term trust even if their underlying blockchain infrastructure changes.

9.3 Soulbound Tokens (SBTs) for On-Chain Representation While our current system stores hashes, representing the credential itself as a Soulbound Token (a non-transferable NFT) offers significant advantages. By issuing an SBT to the student's wallet, the credential becomes a permanent, non-tradable part of their digital identity. This prevents students from transferring or selling their credentials to other wallets while still maintaining the off-chain privacy of the heavy document data.

9.4 Cross-Chain Interoperability Protocols Deploying on a single network (like Polygon or Ethereum) creates vendor lock-in. The next iteration will utilize cross-chain messaging protocols (such as Chainlink CCIP or LayerZero). This will allow an institution to issue a credential on a low-cost Layer 2 network, while a recruiter can verify the authenticity from an entirely different blockchain, ensuring the system remains agnostic and resilient against any single network's failure.

9.5 Native Mobile Identity Wallet Integration The current reliance on browser-based Web3 wallets (like MetaMask) creates friction for non-technical users. The future system will include a dedicated mobile application or integrate directly with OS-level wallets (like Apple Wallet

or Google Wallet). This allows students to hold their Verifiable Credentials directly on their smartphones and present them via NFC or dynamic QR codes for instant, in-person verification at job fairs or border crossings.

9.6 AI-Powered Legacy Document Ingestion To onboard universities quickly, the system needs to handle decades of historical paper records. We will integrate an Optical Character Recognition (OCR) and Machine Learning pipeline. The AI will securely scan legacy paper degrees, extract the metadata, format it into the W3C Verifiable Credential standard, and automatically push the batch to the Merkle tree hashing algorithm for mass retroactive issuance.

10. BIBLIOGRAPHY

1. **World Wide Web Consortium (W3C).** "Verifiable Credentials Data Model v1.1." W3C Recommendation. This is the absolute bible for your project. It defines the exact data model you are using to keep credentials standardized and interoperable.
    - Link: https://www.w3.org/TR/vc-data-model/
2. **MIT Media Lab.** "Blockcerts: The Open Standard for Blockchain Credentials." This is the pioneering project that proved your exact architectural concept. Cite this to show your work aligns with Ivy League research.
    - Link: https://www.blockcerts.org/

Technical Documentation and Infrastructure

3. **Protocol Labs.** "IPFS Documentation: Decentralized Storage." You need this to defend your off-chain storage strategy. It proves you understand how content addressing (CIDs) prevents link rot compared to traditional URLs.
    - Link: https://docs.ipfs.tech/
4. **Ethereum Foundation.** "Smart Contract Security Best Practices." Use this to back up the testing and security claims in Chapter 7. It proves you followed industry-standard threat mitigation.
    - Link: https://consensys.github.io/smart-contract-best-practices/
5. **Nakamoto, S.** "Bitcoin: A Peer-to-Peer Electronic Cash System." (2008). You are using Merkle Trees for batching. You must cite the original paper that popularized Merkle Trees in decentralized ledgers to show you understand the root mathematics of your cost optimization.
    - Link: https://bitcoin.org/bitcoin.pdf

Academic and System Architecture Research

6. **Zheng, Z., et al.** "An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends." *IEEE International Congress on Big Data*. (2017). A standard, highly cited academic paper to ground your general blockchain architecture definitions.
    - Link: https://ieeexplore.ieee.org/document/8029379
7. **Ferdous, M. S., et al.** "Decentralized Identity and Verifiable Credentials." *IEEE Communications Standards Magazine*. (2019). This provides academic weight to your claims about shifting trust from centralized institutions to cryptographic proofs.
    - Link: https://ieeexplore.ieee.org/document/8936913